

Advanced Natural Language Processing

Course information

Grading formula:

Total = Rounding($0.5 * \text{HW} + 0.5 * \text{Exam}$)

6 homework assignments

Links:

Telegram chat: https://t.me/+4D_eKzPBg70wNjQy

GitHub: <https://github.com/ashaba1in/hse-nlp/tree/eng>

About homeworks

All homework will require writing code and a report

Code implementation:

- You can do it as you like
- Better to built it as a project, because
 1. It's easier not to get confused in the code when there's a lot of it
 2. It's easier for us to check structured homework
 3. The project can be posted on git and bragged about

Report:

- PDF document describing the work done
- *We* need it to simplify the check
- *You* need it to learn to summarize your work

План курса

1. Text classification and generation
2. Transformers
3. BERT and GPT
4. Modern Transformer architecture
5. Instruction tuning, Reasoning
6. LLM training pipeline: pre-training, mid-training, post-training
7. Retrieval-Augmented Generation
8. Parameter-Efficient Fine-Tuning, Model steering
9. Quantization, distillation, speculative decoding
10. AI Interpretability and Safety
11. Text diffusion models
12. Multimodal LLMs

Text classification

Types of NLP classification tasks

Binary classification

- Is message a spam or not

Multi-class classification

- How urgently do you need to respond to the client?

Multi-label classification

- What is the topic of the paper?

Word classification

- Named Entity Recognition (NER)
- ~~Text generation (spoilers)~~

Types of NLP classification tasks

Binary classification

- Is message a spam or not

Multi-class classification

- How urgently do you need to respond to the client?

Multi-label classification

- What is the topic of the paper?

Word classification

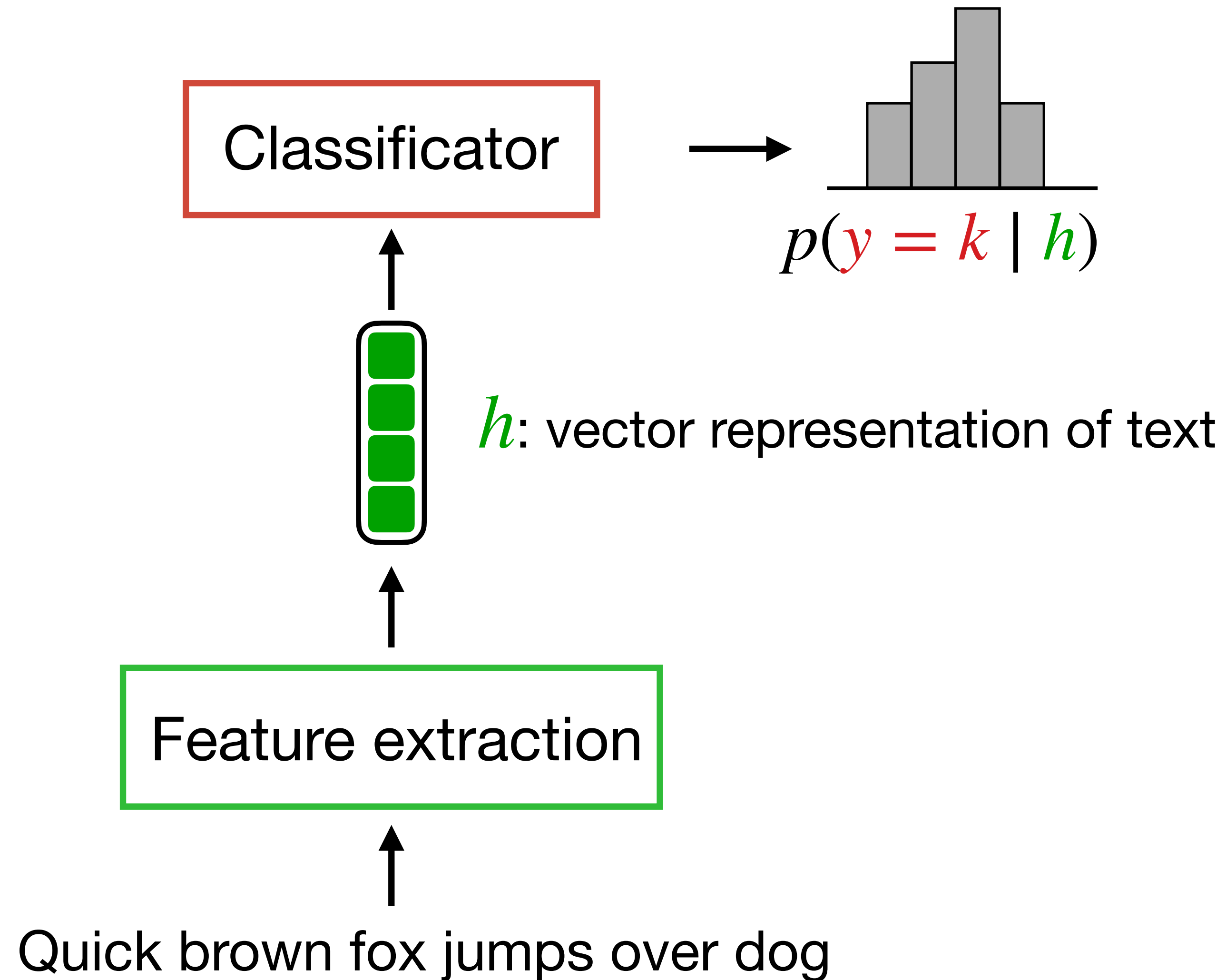
- Named Entity Recognition (NER)
- ~~Text generation (spoilers)~~

All tasks in NLP are
classification

Datasets for classification

Name	Task	Classes	Size	Average Length	Metric
SST	tonality	5 or 2	11,855	19	Accuracy
Yelp	tonality	5 or 2	280,000	179	Accuracy
IMDb	tonality	2	50,000	271	Accuracy
QQP	paraphrasing	2	404,291	22	F1 / Accuracy
CoLA	grammaticality	2	10,657	9	Matthew's Corr
AG News	topic	4	120,000	44	Accuracy
Yahoo! Answers	topic	10	1,400,000	131	Accuracy
DBpedia	topic	14	560,000	67	Accuracy

General model scheme



The main NLP task is feature extraction

Short NLP history

Year	Method	Target
1957	distributional hypothesis (J.R. Firth)	—
1972	TF-IDF	Word relevance ranking
1990	Latent semantic analysis (LSA)	Word semantic proximity
2003	Latent Dirichlet allocation (LDA)	Topic modeling
2013	word2vec	Word embeddings
2016	fastText	Fast text classification Solving out-of-vocabulary problem
2018	contextual representations	Homonymy, word order

Text representation

Text is always represented as a sequence of simple entities
(words/tokens)

~~$x = \text{"Quick brown fox jumps over dog"}$~~

$x = [\text{"Quick"}, \text{"brown"}, \text{"fox"}, \text{"jumps"}, \text{"over"}, \text{"dog"}]$

Naive Bayes

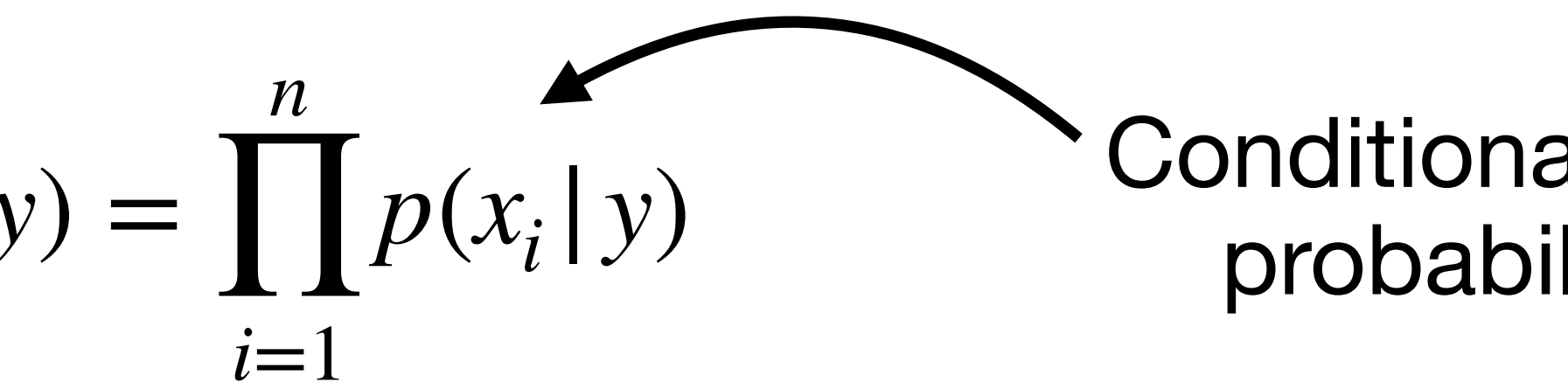
$$y = \arg \max p(y | x) = \arg \max \frac{p(x | y) \cdot p(y)}{p(x)} = \arg \max p(x | y)p(y)$$

Naive Bayes

$$y = \arg \max p(y | x) = \arg \max \frac{p(x | y) \cdot p(y)}{p(x)} = \arg \max p(x | y)p(y)$$

$$p(x | y) = \prod_{i=1}^n p(x_i | y)$$

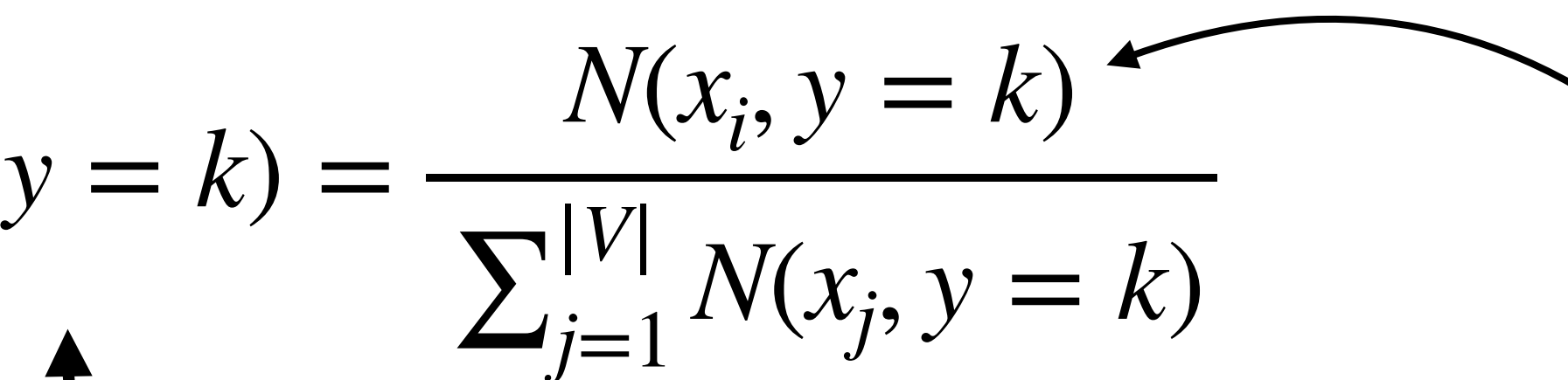
Conditional text probability



A curved arrow points from the text 'Conditional text probability' to the product symbol in the equation above.

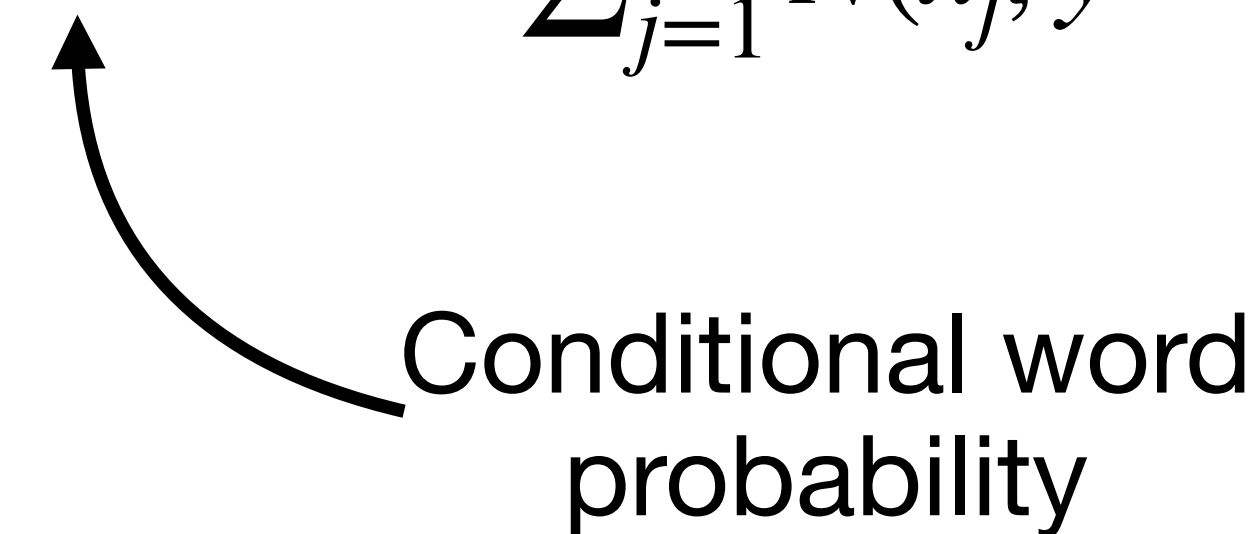
$$p(x_i | y = k) = \frac{N(x_i, y = k)}{\sum_{j=1}^{|V|} N(x_j, y = k)}$$

Word count



A curved arrow points from the text 'Word count' to the numerator $N(x_i, y = k)$ in the equation above.

Conditional word probability



A curved arrow points from the text 'Conditional word probability' to the equation $p(x_i | y = k)$ in the block above.

$$p(y = k) = \frac{1}{N} \sum_{i=1}^N [y_i = k]$$

Class probability



A curved arrow points from the text 'Class probability' to the equation $p(y = k)$ in the block above.

Naive Bayes

$$y = \arg \max p(y | x) = \arg \max \frac{p(x | y) \cdot p(y)}{p(x)} = \arg \max p(x | y)p(y)$$

$$p(x | y) = \prod_{i=1}^n p(x_i | y)$$

Conditional text probability

$$p(x_i | y = k) = \frac{N(x_i, y = k)}{\sum_{j=1}^{|V|} N(x_j, y = k)}$$

Word count

Conditional word probability

$$p(y = k) = \frac{1}{N} \sum_{i=1}^N [y_i = k]$$

Class probability

What are its disadvantages?

TF-IDF + Logistic regression

TF-IDF

$$\text{tf}(t, d) = \frac{1}{N_d} \sum_{i=1}^{N_d} [t = d_i]$$

$$\text{idf}(t, D) = \log \frac{|D|}{\sum_{d \in D} [t \in d]}$$

$$\text{tf-idf}(t, d, D) = \text{tf}(t, d) \cdot \text{idf}(t, D)$$

TF-IDF + Logistic regression

TF-IDF

$$\text{tf}(t, d) = \frac{1}{N_d} \sum_{i=1}^{N_d} [t = d_i]$$

$$\text{idf}(t, D) = \log \frac{|D|}{\sum_{d \in D} [t \in d]}$$

$$\text{tf-idf}(t, d, D) = \text{tf}(t, d) \cdot \text{idf}(t, D)$$

$\text{text} = [\text{"Quick"}, \text{"brown"}, \text{"fox"}, \text{"jumps"}]$

$$x = \{\text{tf-idf}(t, \text{text}, D)\}_{t \in \text{Vocab}}$$

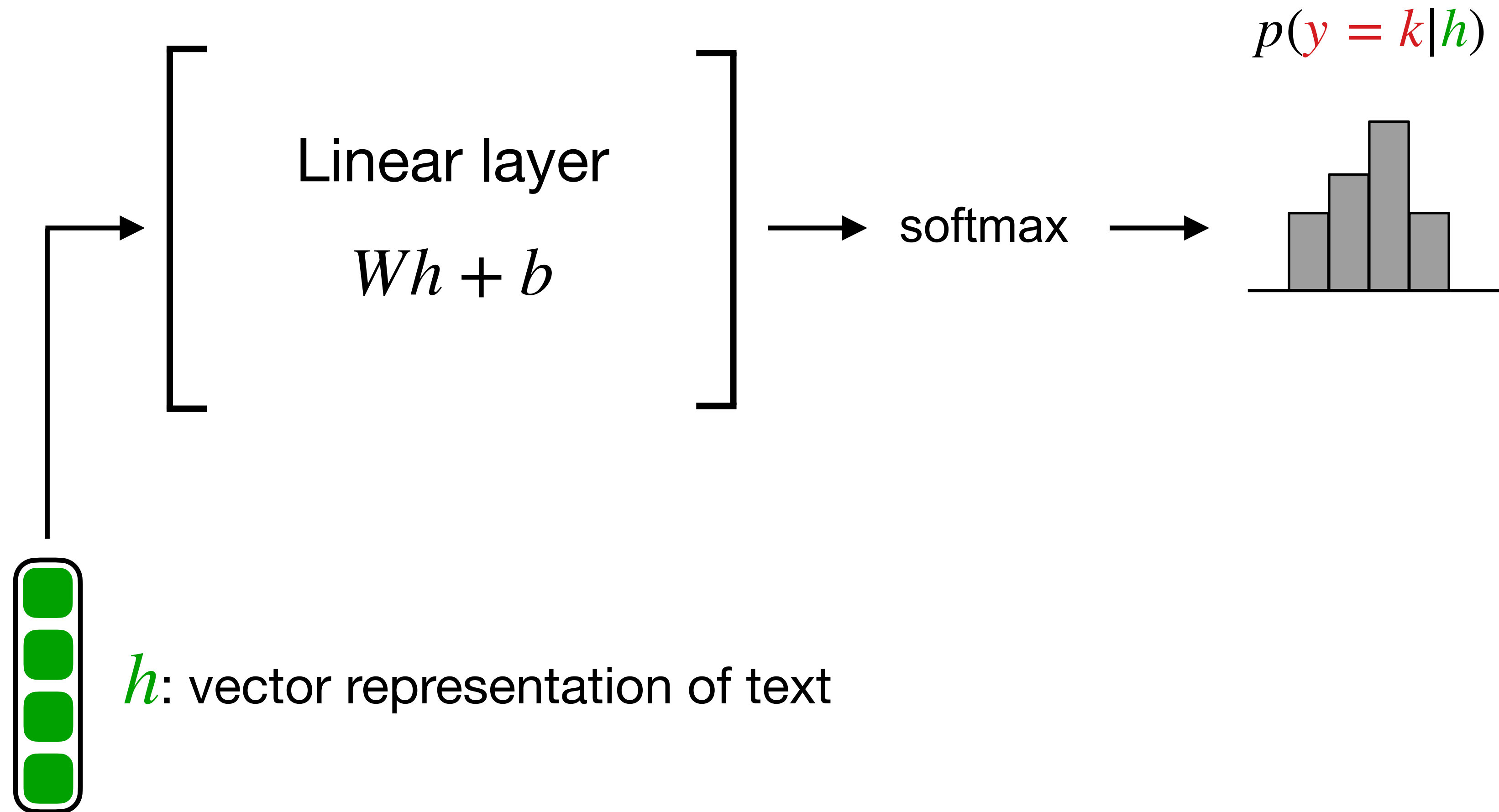
Logistic regression

$$p_y = \text{softmax}(Wx + b)_y$$

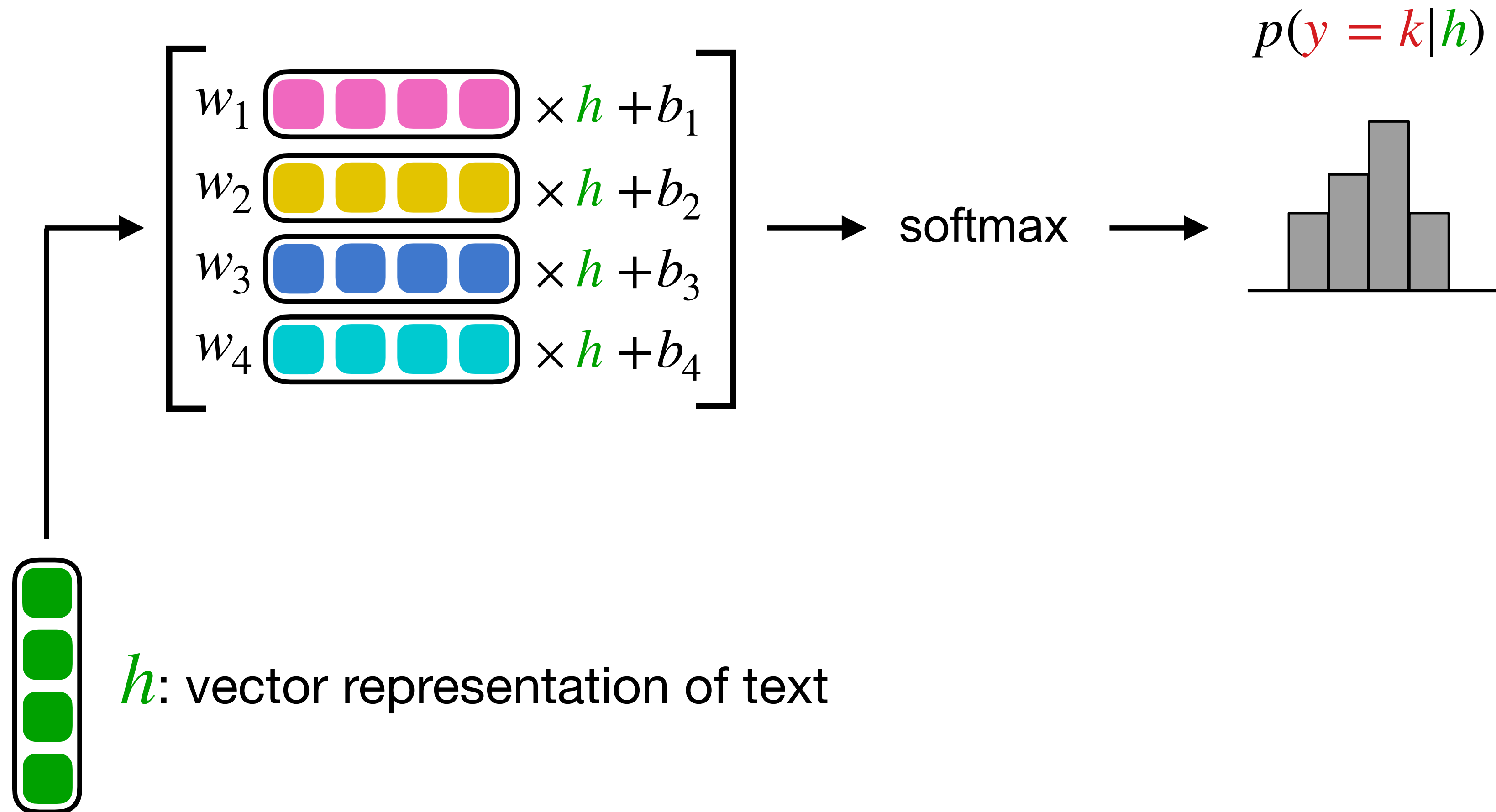
W, b – trainable parameters

$$L(W, b) = - \sum_{i=1}^N \sum_{k=1}^K [y = k] \cdot \log p_k$$

Linear layer



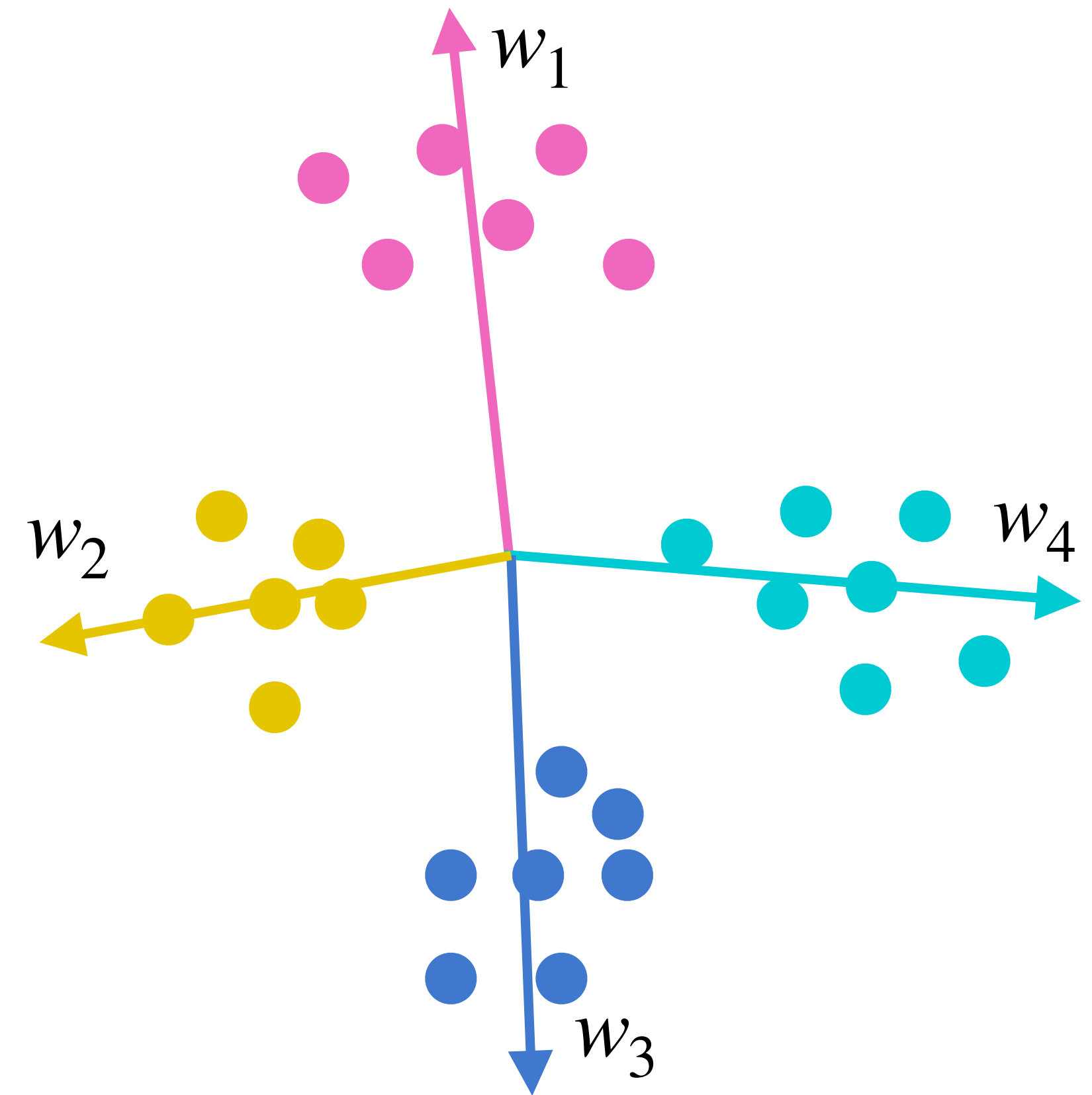
Linear layer



Linear layer

The vectors of the linear layer for each class should be correlated with the vector representations of the class elements.

The scalar product of the vectors is **maximal** when they are **co-directed**.



Disadvantages of approaches

- They don't take into account the relationship between words
- They don't take into account the word order

$p(y = + \mid \text{It is good, not bad})$

$\parallel$

$p(y = + \mid \text{It is bad, not good})$

- Features are extracted manually

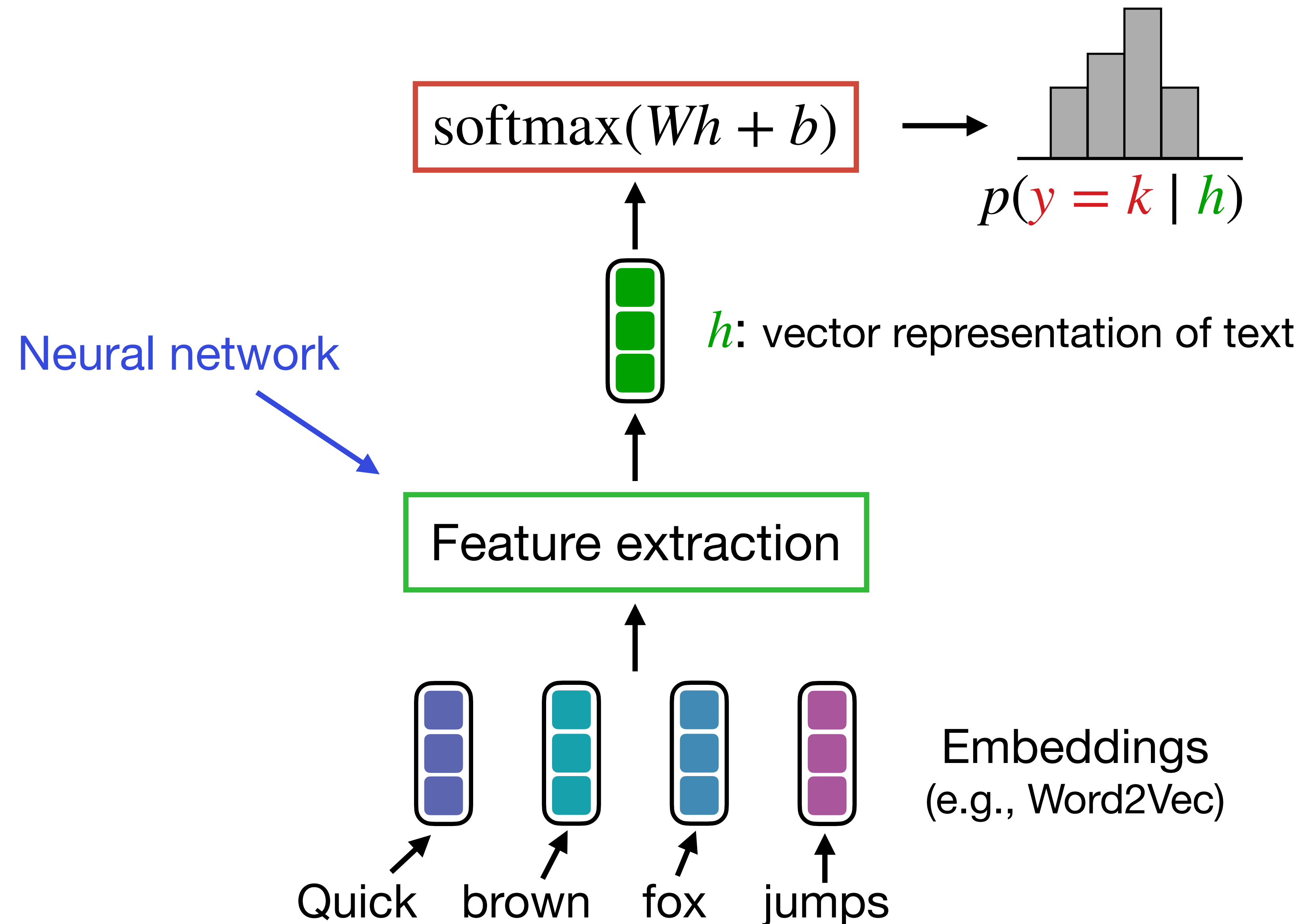
Advantages of approaches

- Fairly good at simple tasks
- Working speed
- Learning time
- Interpretability

Interpretability is very important when the cost of error is high

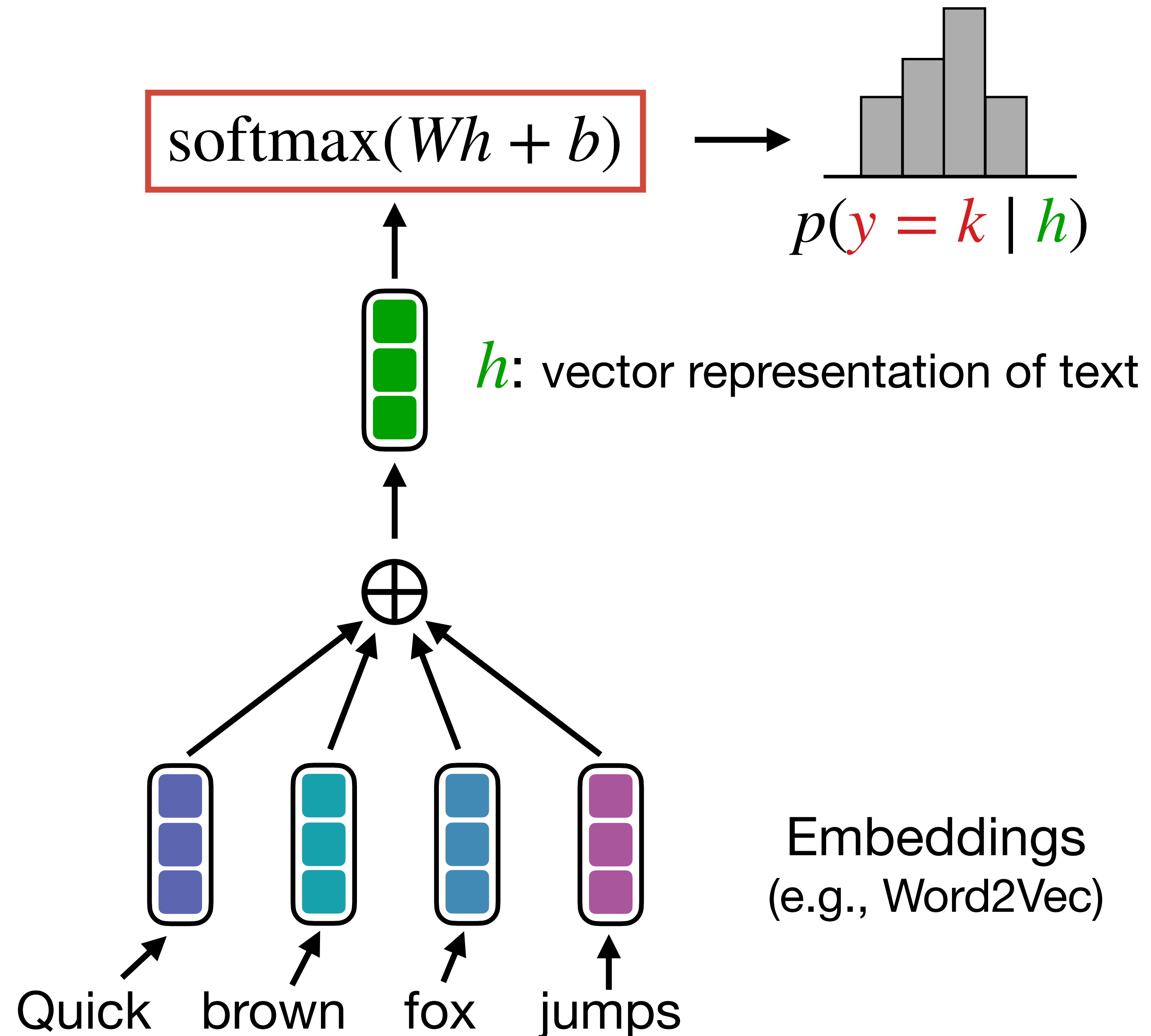
- Medical diagnosis
- Sentencing in court

Neural networks



Bag of Embeddings

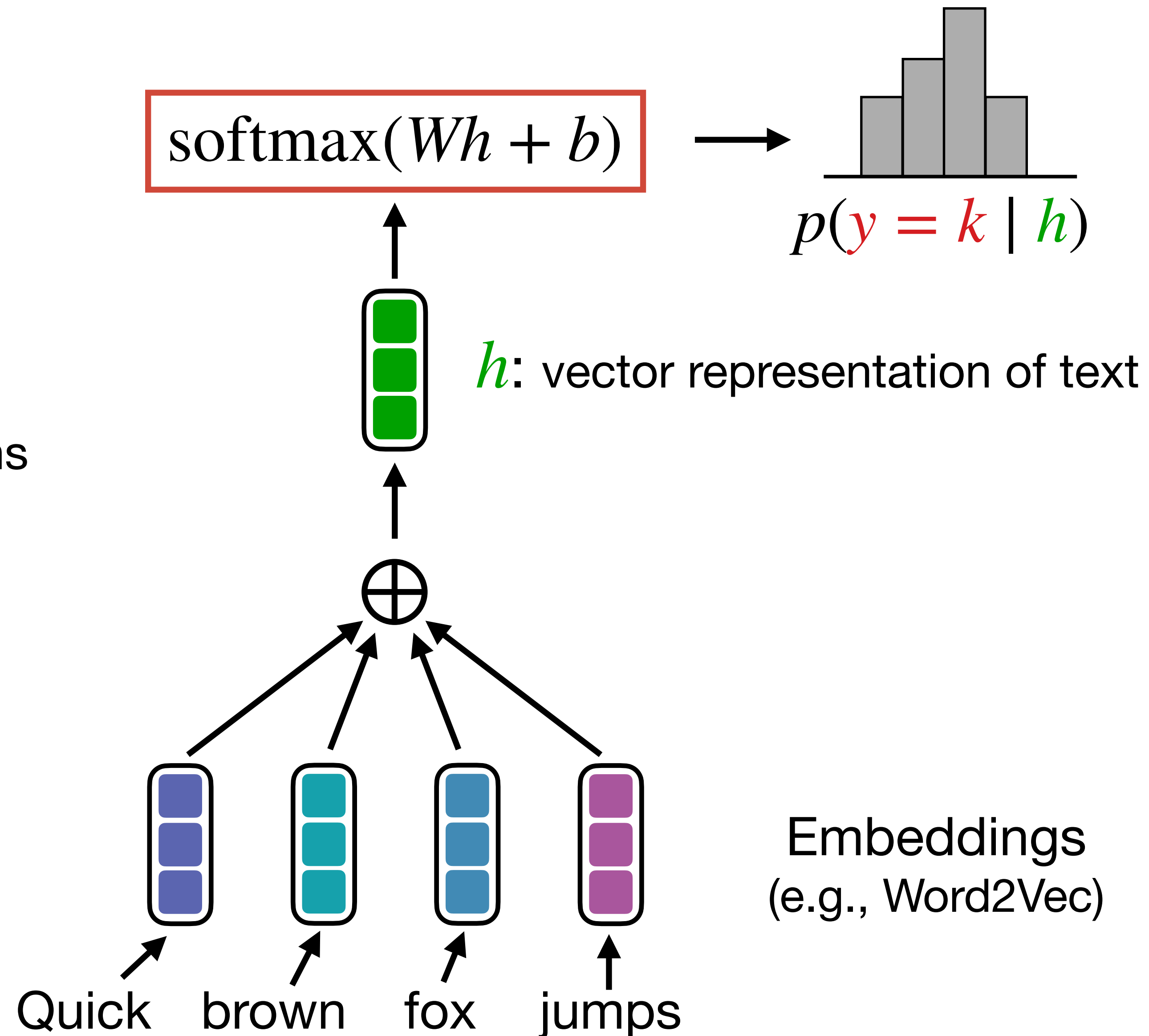
Represent text as a sum of embeddings



Bag of Embeddings

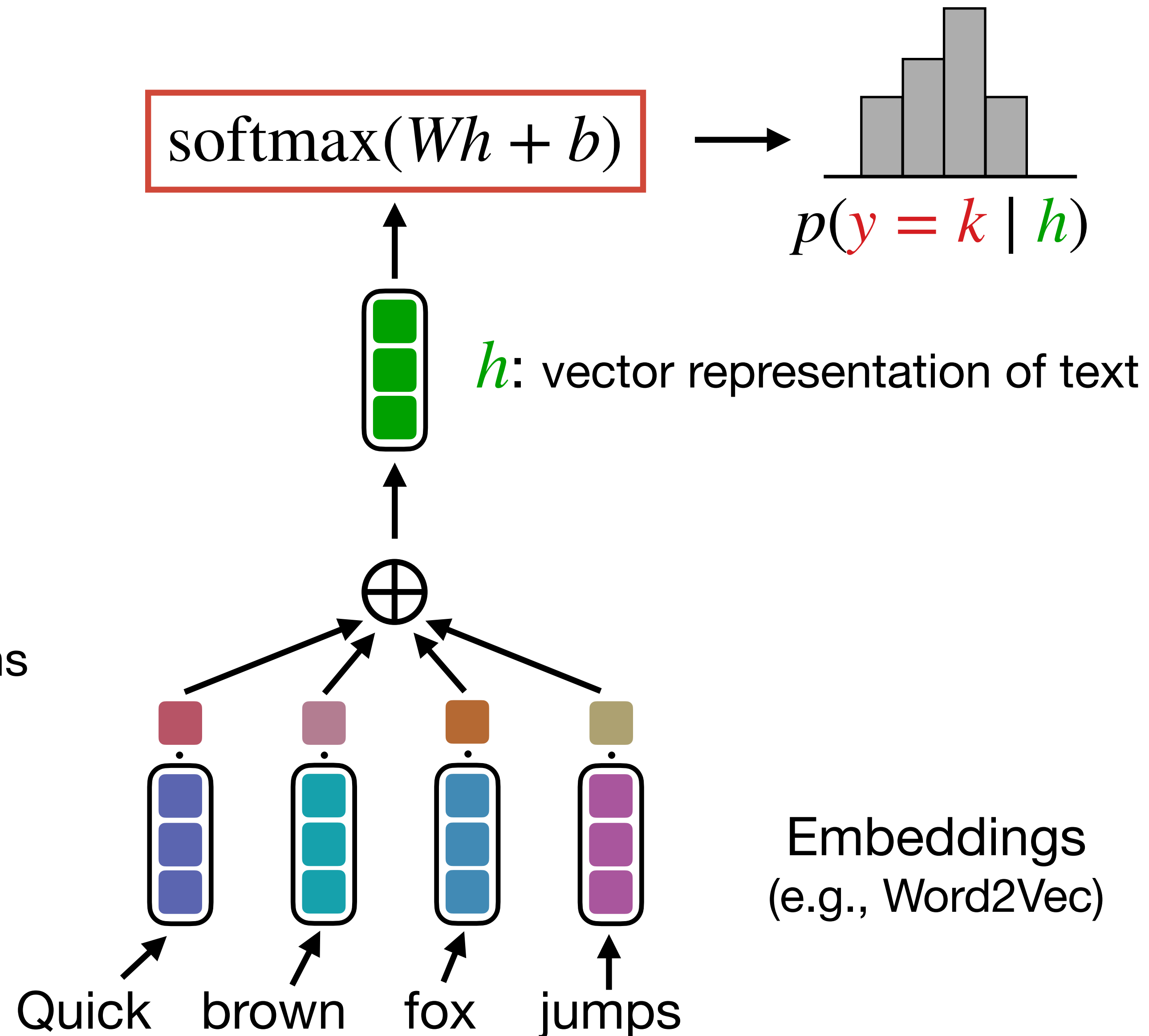
Represent text as a sum of embeddings

- + Very easy to implement
- Do not consider word relations
- Neutral words have large cumulative weight



Weighted Bag of Embeddings

- Multiply embeddings by TF-IDF weights
 - Sum up all embeddings
- + Still easy to implement
- + Less important words have lower weight
- Do not consider word relations



Text generation

What generation is used for?

Word auto-completion

Quick red fox jumps

Phrase auto-completion

Why do birds fly
sing
fly in a wedge

Dialog systems

– What is the weather in Moscow?

It's 15 degrees Celsius in Moscow –

Problem statement

The text must meet the following requirements:

- Logical coherence
- Compliance with language norms

We can try to set the rules manually.

But there are too many rules, so nothing good will come of it.

Problem statement

We will learn to **mimic** human speech

To do this, we will learn to evaluate the probability of texts

$$p(\text{"Quick brown fox jumps"}) = p(\text{"Quick"}, \text{"brown"}, \text{"fox"}, \text{"jumps"})$$

Problem statement

We will learn to **mimic** human speech

To do this, we will learn to evaluate the probability of texts

$$p(\text{"Quick brown fox jumps"}) = p(\text{"Quick"}, \text{"brown"}, \text{"fox"}, \text{"jumps"}) = ???$$

Problem statement

We will learn to **mimic** human speech

To do this, we will learn to evaluate the probability of texts

$$p(\text{"Quick brown fox jumps"}) = p(\text{"Quick"}, \text{"brown"}, \text{"fox"}, \text{"jumps"}) = ???$$

You can use chain rule and generate text word by word

$$p(x_1, \dots, x_m) = \prod_{i=1}^m p(x_i | x_{<i})$$

How to estimate $p(x_i | x_{<i})$?

From Naive Bayes

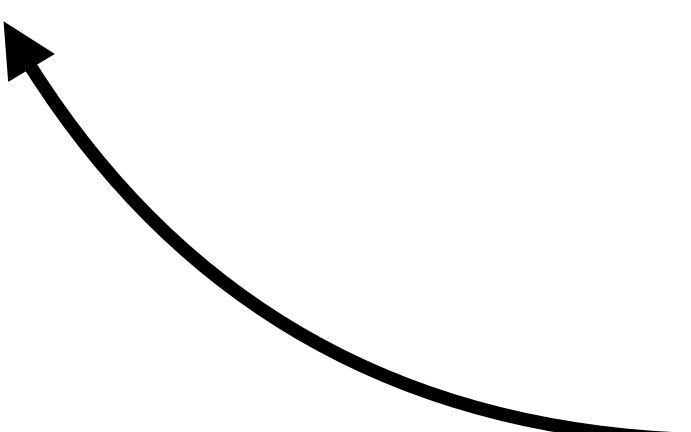
$$p(x_i | y = k) = \frac{N(x_i, y = k)}{\sum_{j=1}^{|V|} N(x_j, y = k)}$$

How to estimate $p(x_i | x_{<i})$?

From Naive Bayes

$$p(x_i | y = k) = \frac{N(x_i, y = k)}{\sum_{j=1}^{|V|} N(x_j, y = k)}$$

Arbitrary
condition length

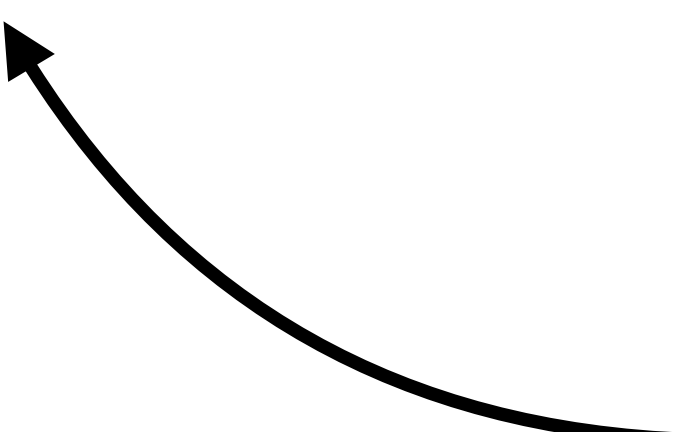


How to estimate $p(x_i | x_{<i})$?

From Naive Bayes

$$p(x_i | y = k) = \frac{N(x_i, y = k)}{\sum_{j=1}^{|V|} N(x_j, y = k)}$$

Arbitrary
condition length



Let's simplify the task - we'll only look at the previous n words.

$$p(x_1, \dots, x_m) \approx \prod_{i=1}^m p(x_i | x_{i-1}, \dots, x_{i-n})$$

This model is called n-gram.

N-gram generation model

We assume that the next word depends only on the previous ones

$$p(x_1, \dots, x_m) \approx \prod_{i=1}^m p(x_i | x_{i-1}, \dots, x_{i-n})$$

If a word has less than n of the previous ones, we fill in the gaps with the <PAD> token

$$\begin{aligned} p(\text{Quick, brown, fox, jumps}) = & \\ & p(\text{Quick} | \text{<PAD>, <PAD>}) \\ & \cdot p(\text{brown} | \text{<PAD>, Quick}) \\ & \cdot p(\text{fox} | \text{Quick, brown}) \\ & \cdot p(\text{jumps} | \text{brown, fox}) \end{aligned}$$

N-gram generation model: training

- We need to estimate the probability $p(x_i | x_{i-1}, \dots, x_{i-n})$.
- Let's manually count how many times x_i occurred after $x_{i-1}, \dots, x_{i-n}$.

$$p(x_i | x_{i-1}, \dots, x_{i-n}) = \frac{p(x_i, x_{i-1}, \dots, x_{i-n})}{p(x_{i-1}, \dots, x_{i-n})} = \frac{N(x_i, x_{i-1}, \dots, x_{i-n})}{N(x_{i-1}, \dots, x_{i-n})}$$

N-gram generation model: generation

Generate words one by one according to probabilities.

N-gram generation model: generation

Generate words one by one according to probabilities.

<PAD> <PAD> _

$$p(\text{Quick} \mid \text{<PAD> <PAD>}) = 0.3$$

$$p(\text{My} \mid \text{<PAD> <PAD>}) = 0.2$$

$$p(\text{When} \mid \text{<PAD> <PAD>}) = 0.15$$

N-gram generation model: generation

Generate words one by one according to probabilities.

<PAD> <PAD> Quick _

$$p(\text{fox} \mid \text{<PAD> Quick}) = 0.34$$

$$p(\text{brown} \mid \text{<PAD> Quick}) = 0.23$$

$$p(\text{bird} \mid \text{<PAD> Quick}) = 0.09$$

N-gram generation model: generation

Generate words one by one according to probabilities.

<PAD> <PAD> Quick brown _

$$p(\text{cat} \mid \text{Quick brown}) = 0.4$$

$$p(\text{fox} \mid \text{Quick brown}) = 0.23$$

$$p(\text{dog} \mid \text{Quick brown}) = 0.09$$

N-gram generation model: generation

Generate words one by one according to probabilities.

<PAD> <PAD> Quick brown fox

Advantages of n-gram model

Advantages of n-gram model

- Texts consist of existing n-grams
- Therefore, the sentences are grammatically correct
- The model is easy to implement and very fast.

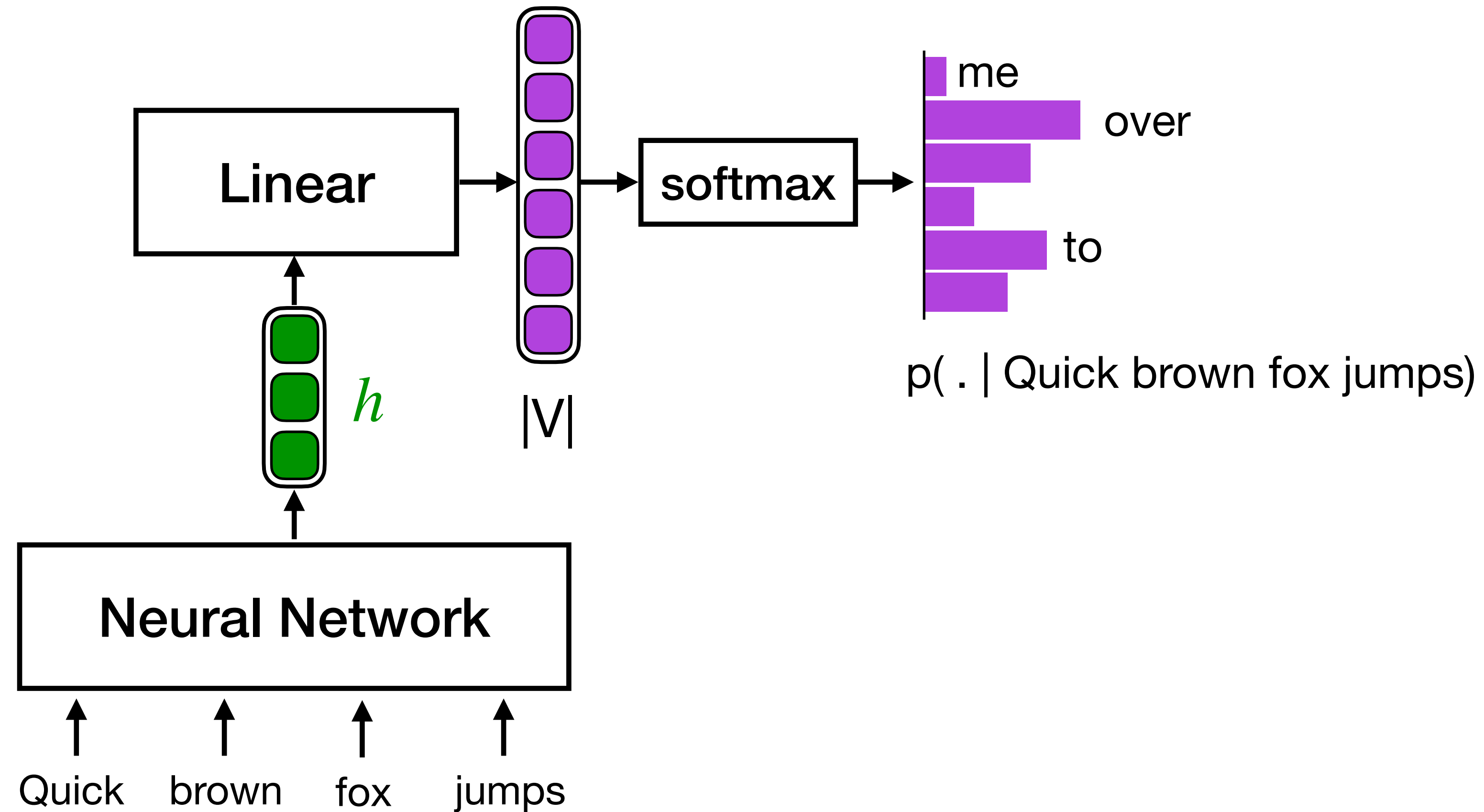
Disadvantages of n-gram model

Disadvantages of n-gram model

- When generating, it looks only at the last n words
- This results in logically incoherent texts
- As n increases, the probabilities of words are estimated worse
- Due to the large size of the dictionary, many n-grams are very rare

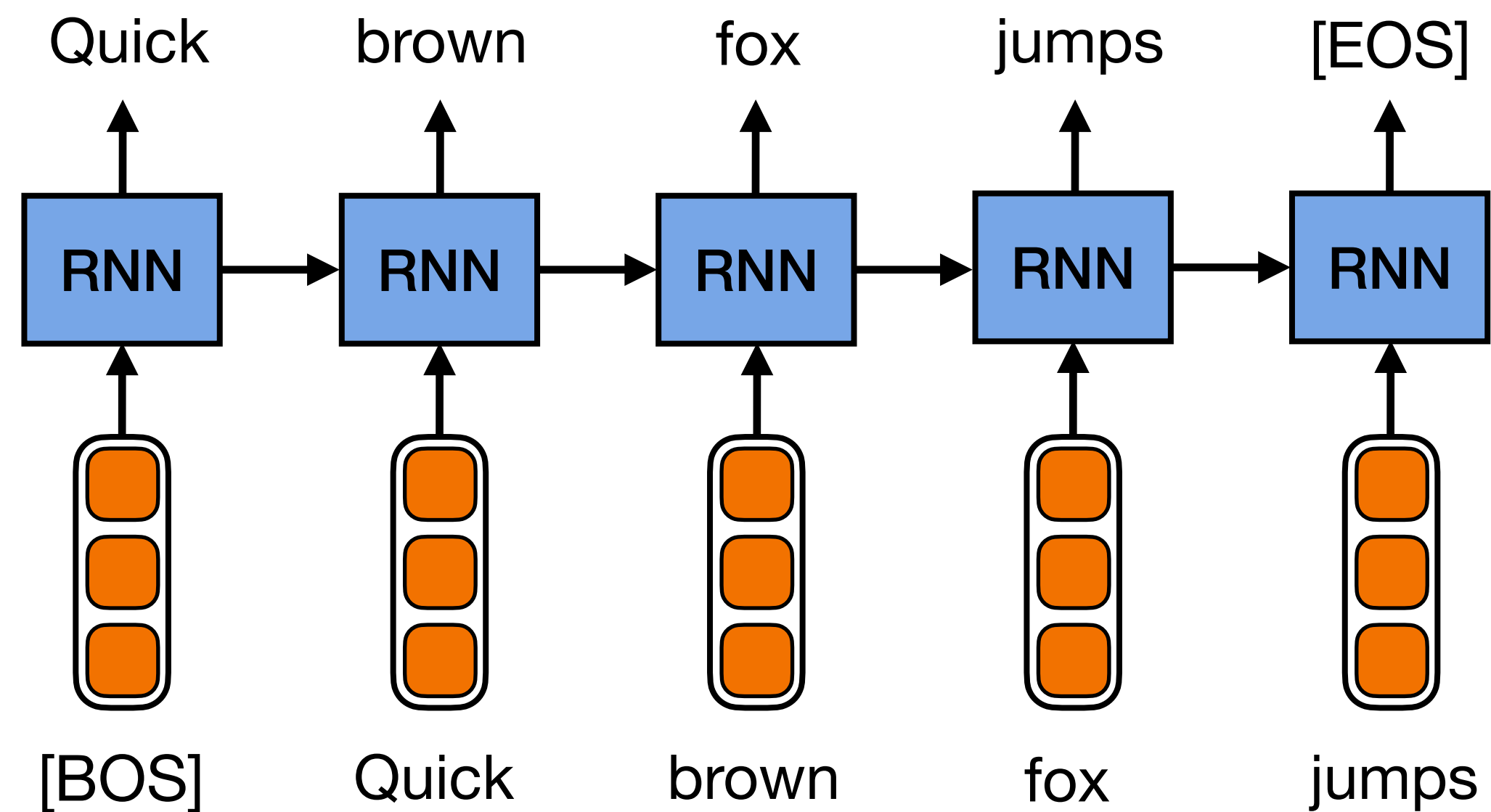
Recurrent Neural Network (RNN)

Neural networks for text generation



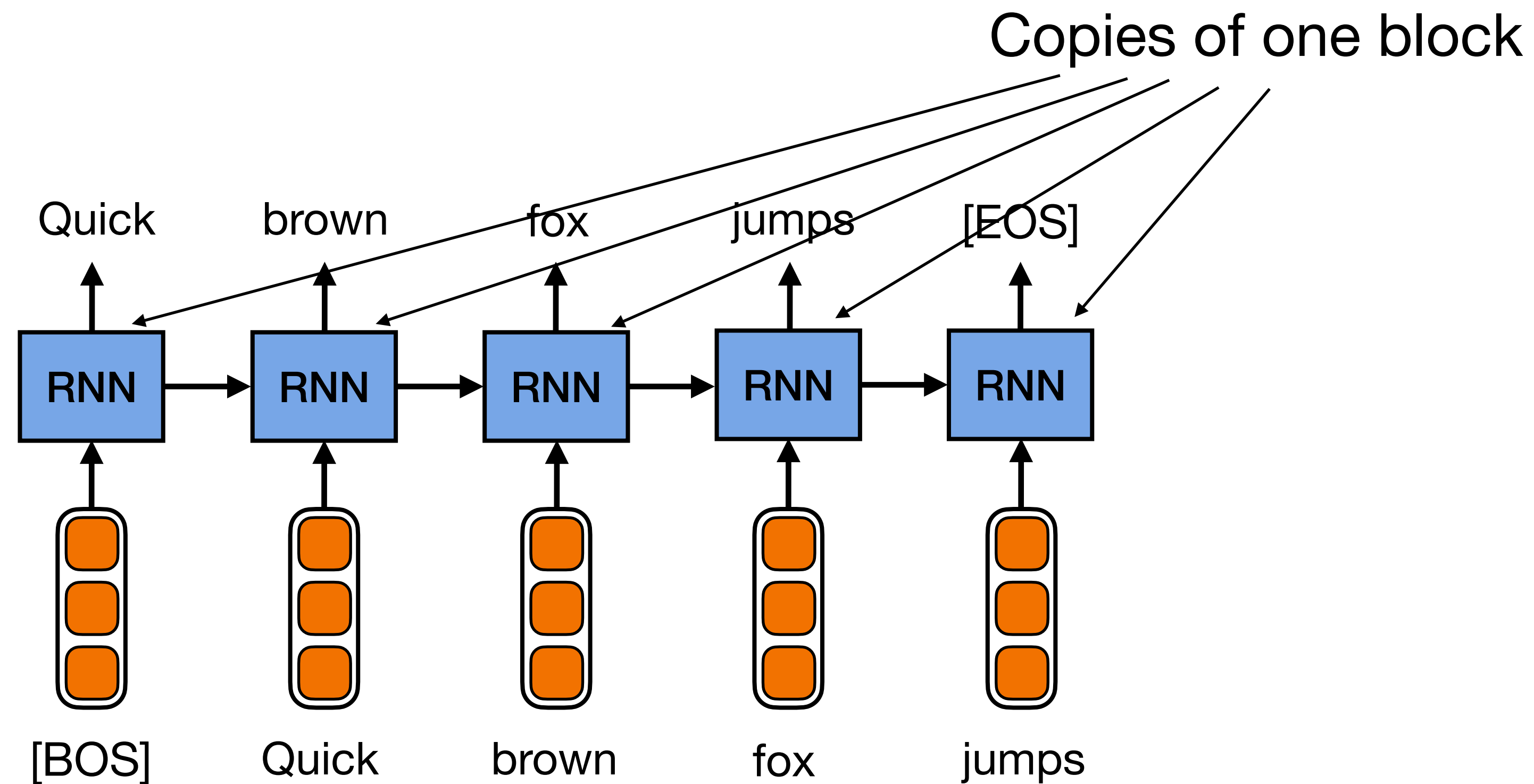
Recurrent neural networks (RNN)

- Designed to work with sequential data
- Each block predicts the next token



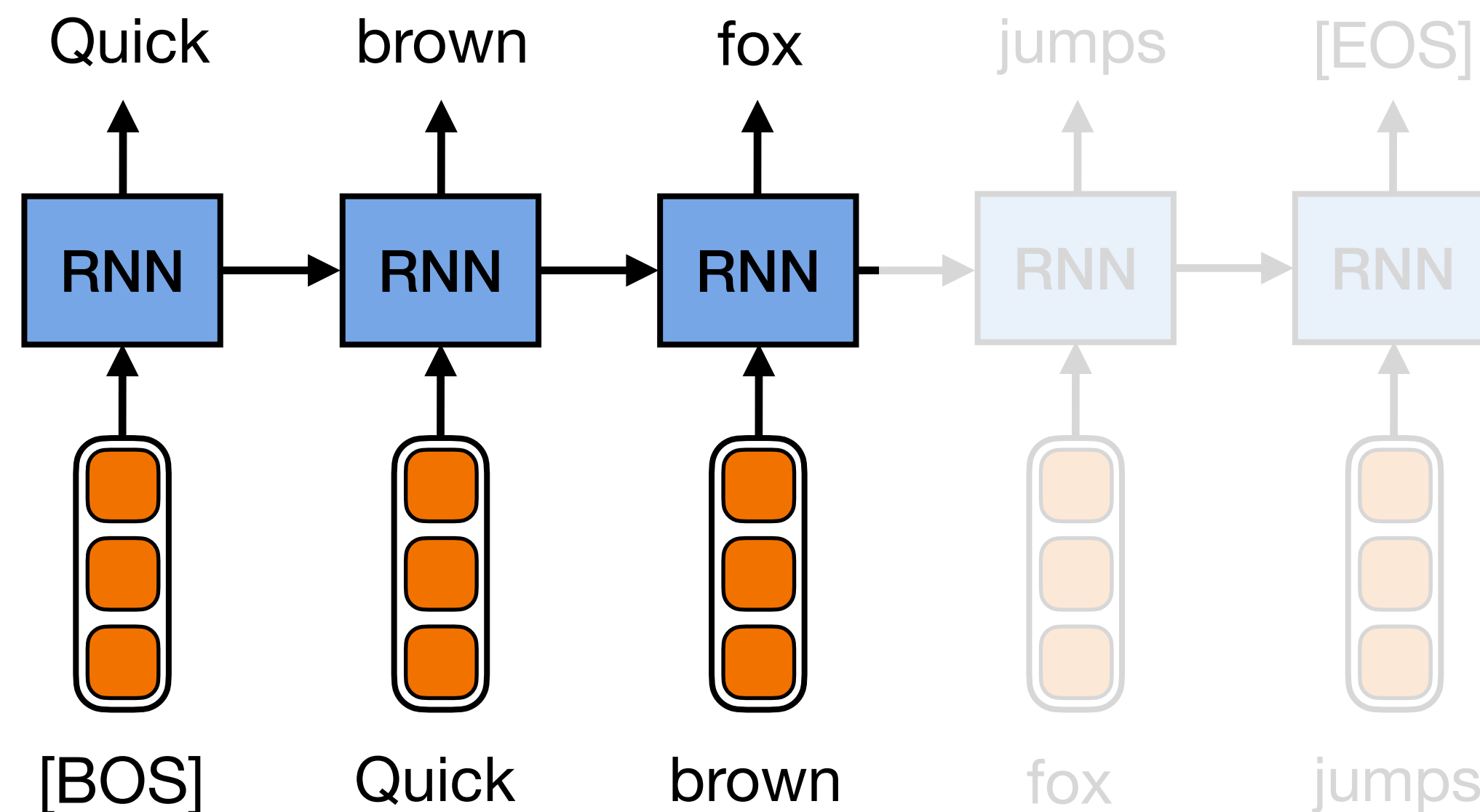
Recurrent neural networks (RNN)

- Designed to work with sequential data
- Each block predicts the next token

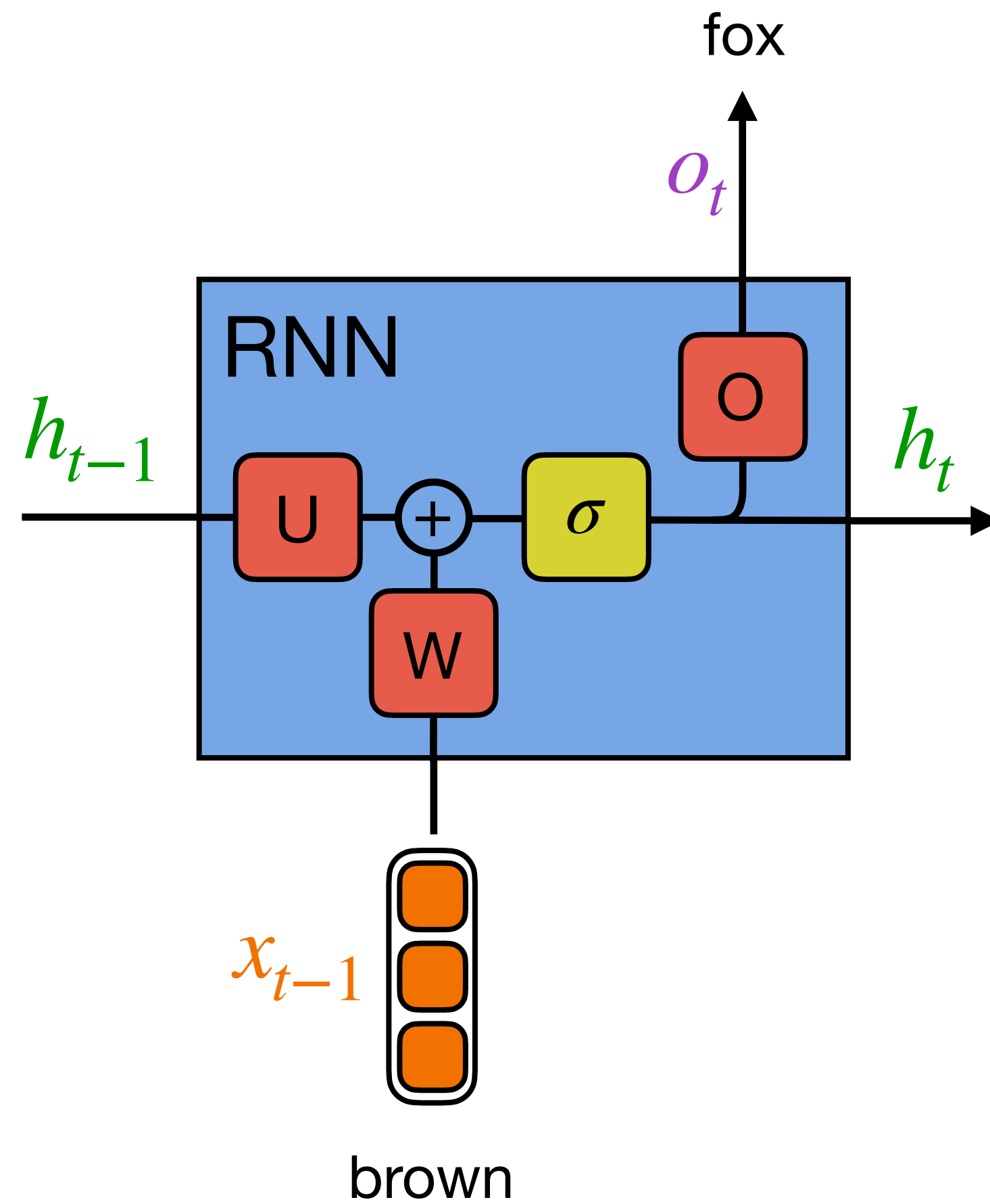


Recurrent neural networks (RNN)

- Designed to work with sequential data
- Each block predicts the next token
- The generation process is intuitive



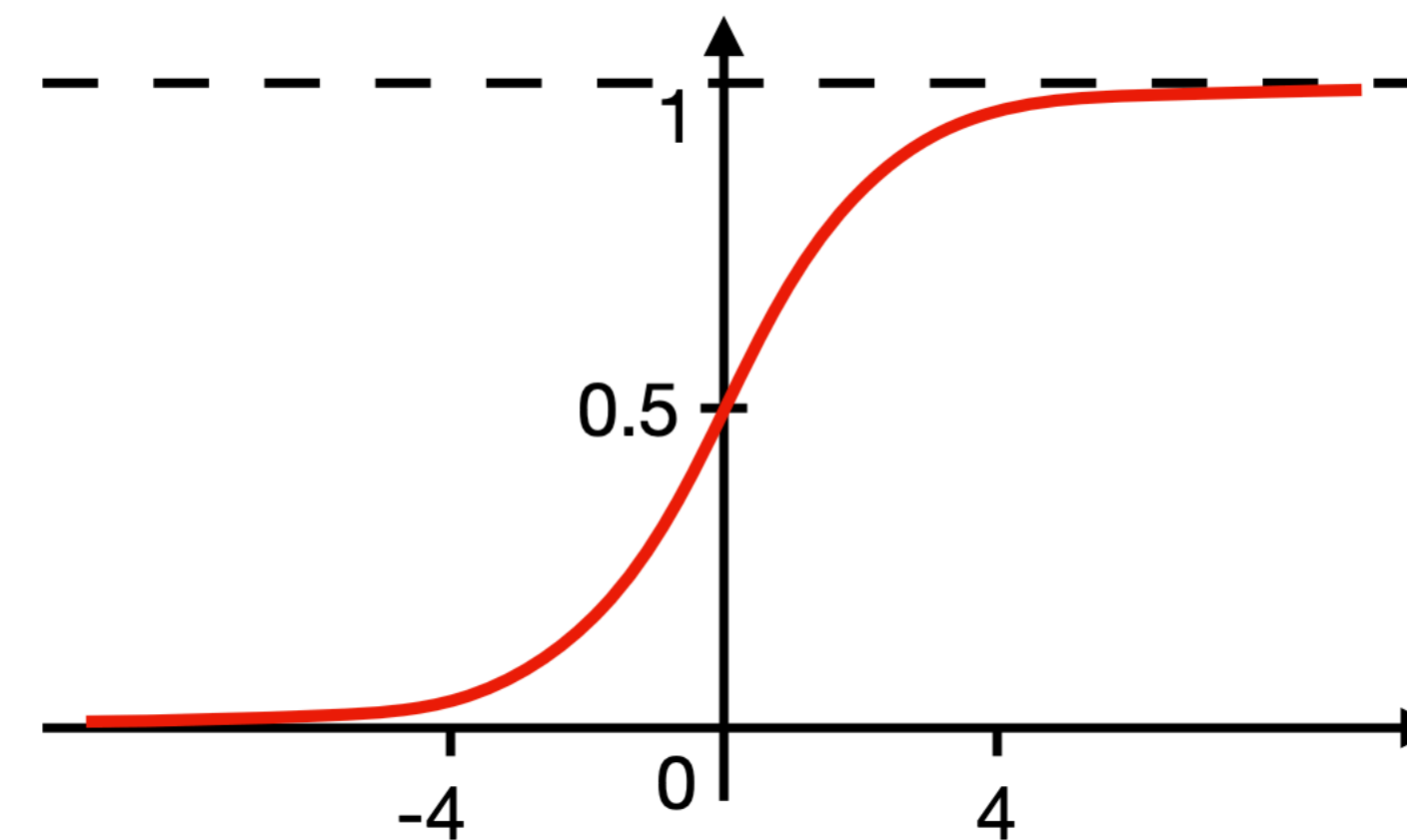
RNN block



$$h_t = \sigma(Wx_{t-1} + Uh_{t-1} + b_h)$$

$$o_t = Oh_t + b_o$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



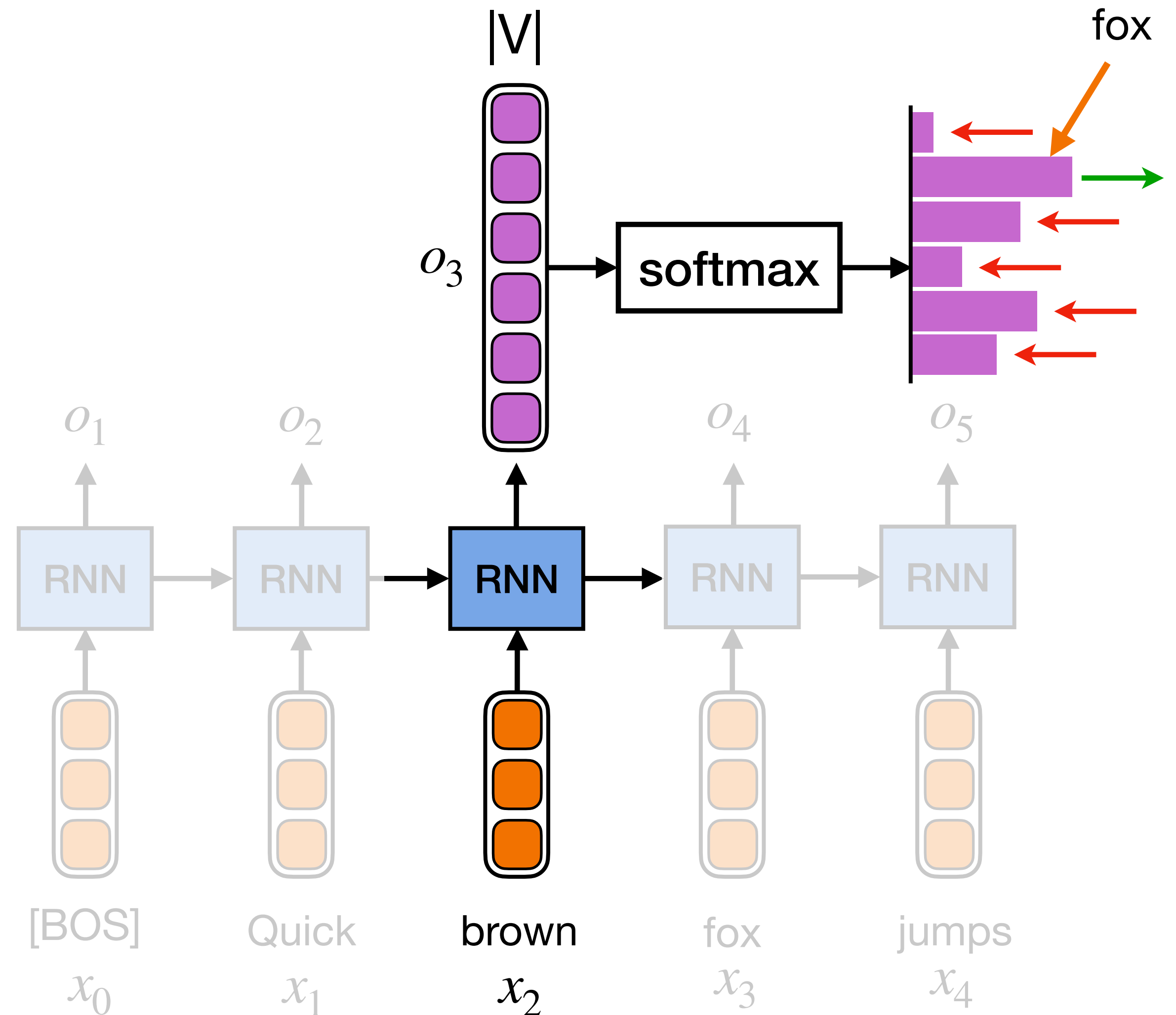
RNN: training

$$p(x_1, \dots, x_m) = \prod_{t=1}^m p(x_t | x_{<t}) \rightarrow \max$$

$$p(\cdot | x_{<t}) = \text{softmax}(o_t)$$

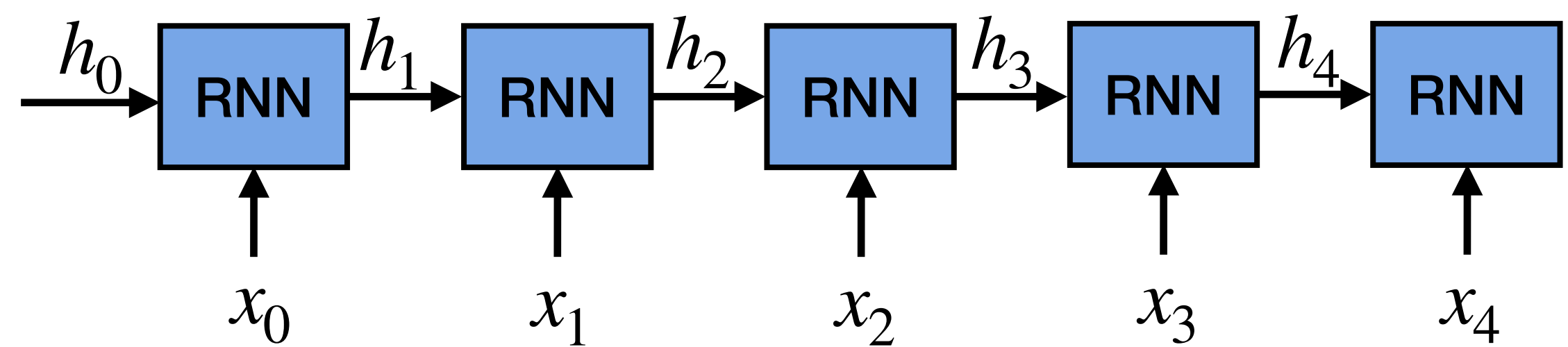
Loss function

$$L(X) = -\frac{1}{|X|} \sum_{x \in X} \sum_{t=1}^m \log p(x_t | x_{<t}) \rightarrow \min$$



RNN gradients

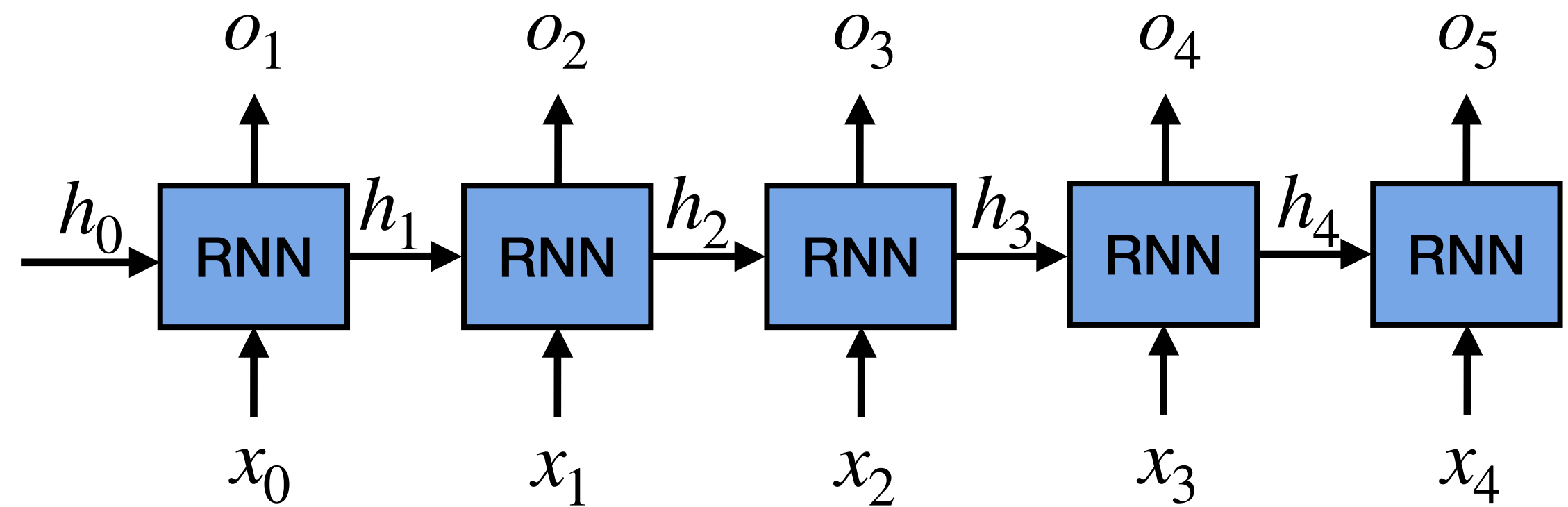
$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$



RNN gradients

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Oh_t + b_o$$

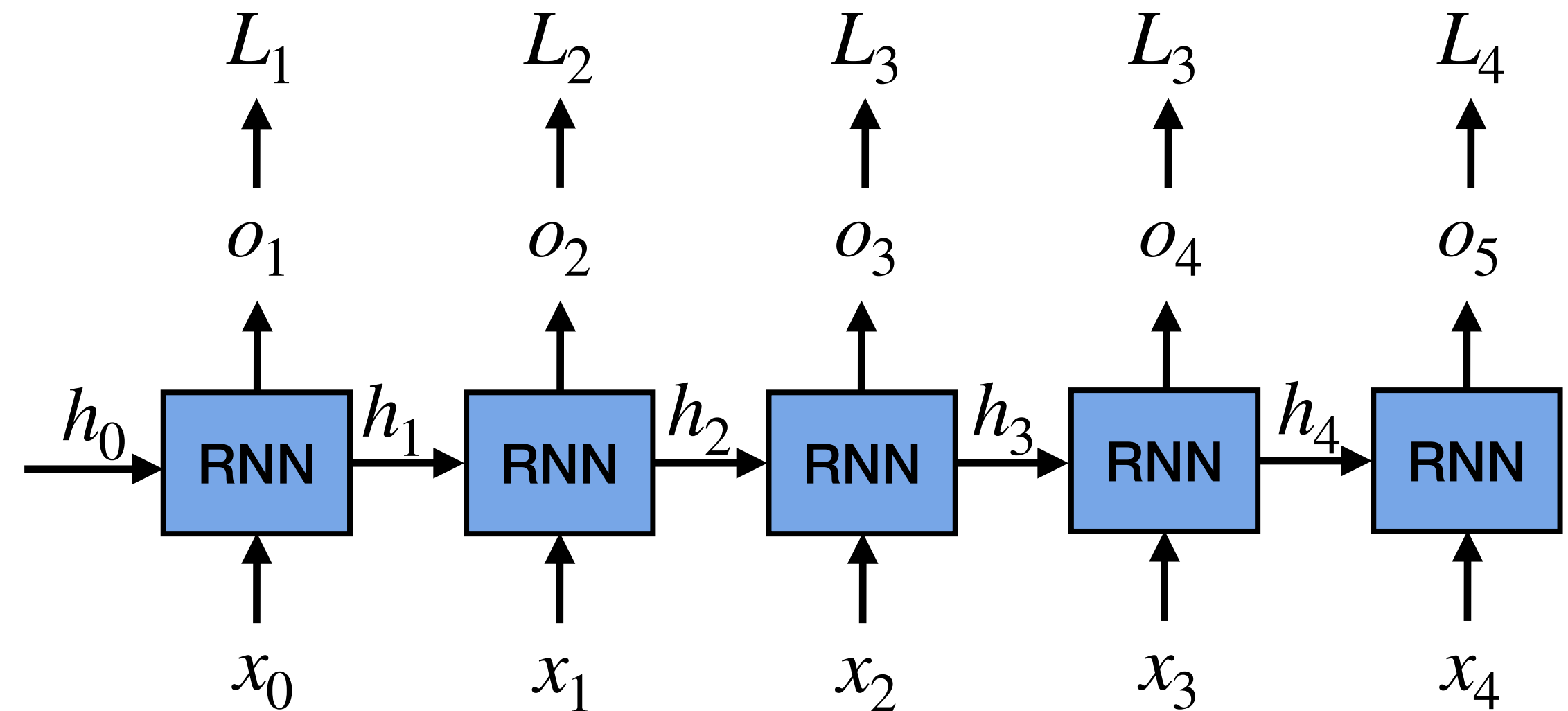


RNN gradients

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Oh_t + b_o$$

$$L_t = -\log p(x_t | x_{<t}) = -\log \text{softmax}(o_t)_{x_t}$$



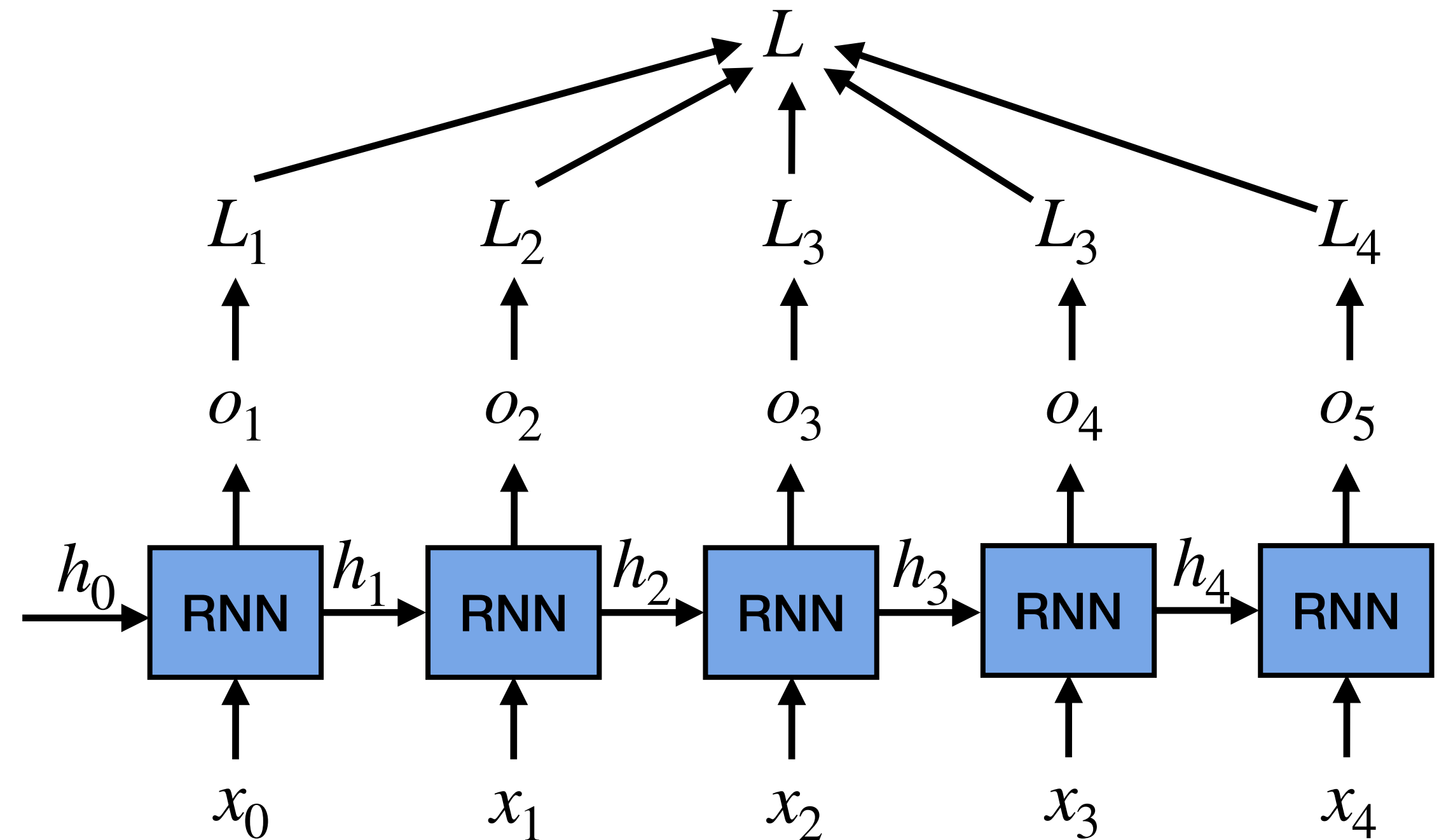
RNN gradients

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Oh_t + b_o$$

$$L_t = -\log p(x_t | x_{<t}) = -\log \text{softmax}(o_t)_{x_t}$$

$$L = \sum_{t=1}^m L_t$$



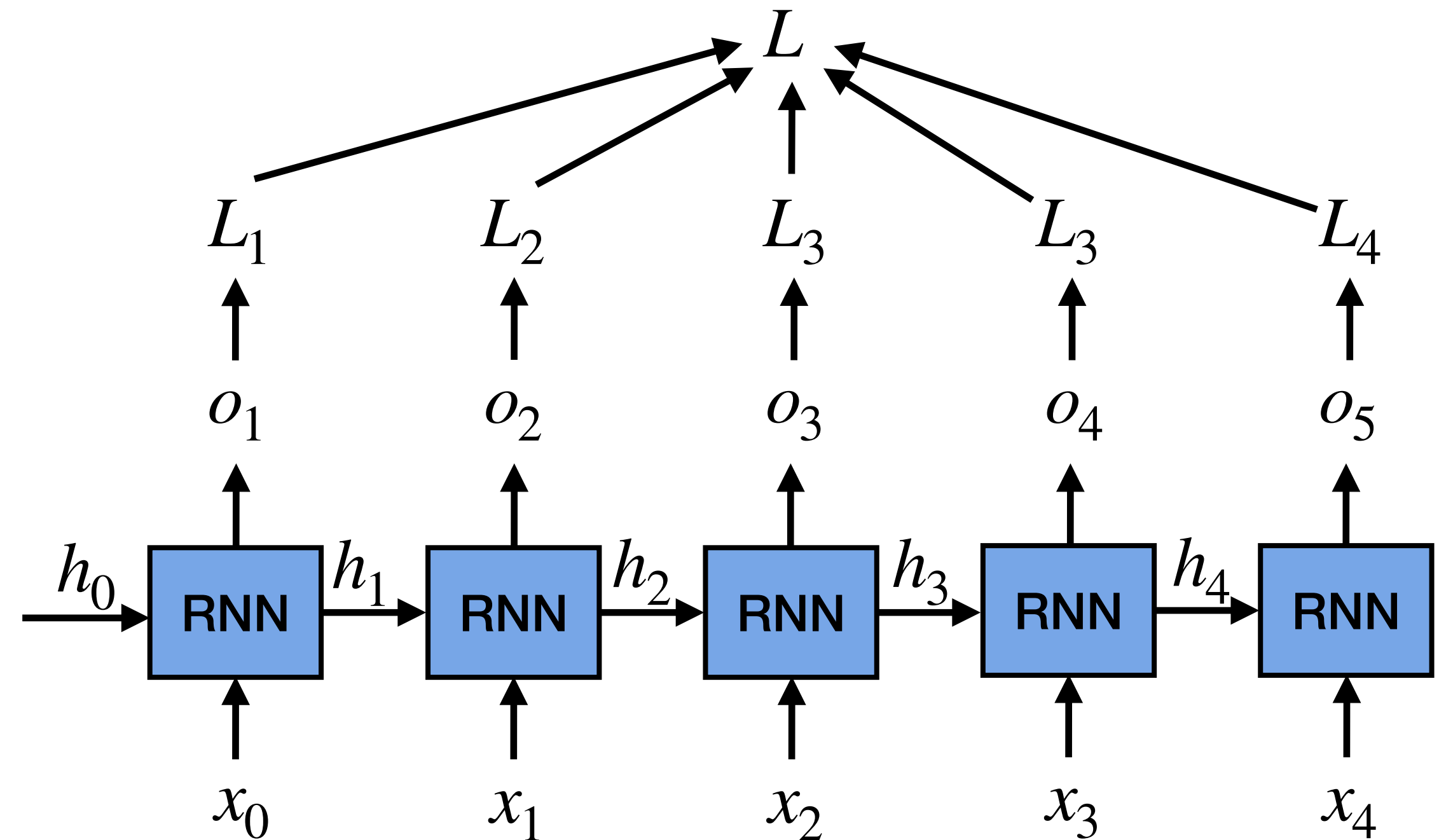
RNN gradients

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Oh_t + b_o$$

$$L_t = -\log p(x_t | x_{<t}) = -\log \text{softmax}(o_t)_{x_t}$$

$$L = \sum_{t=1}^m L_t$$



Let's see how gradients behave. It's going to hurt a little.

RNN gradients

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Oh_t + b_o$$

$$L_t = -\log p(x_t | x_{<t}) = -\log \text{softmax}(o_t)_{x_t}$$

$$L = \sum_{t=1}^m L_t$$

chain rule

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \frac{dh_t}{dU}$$

RNN gradients

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Oh_t + b_o$$

$$L_t = -\log p(x_t | x_{<t}) = -\log \text{softmax}(o_t)_{x_t}$$

$$L = \sum_{t=1}^m L_t$$

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \frac{dh_t}{dU}$$

$$\frac{dh_t}{dU} = \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{dh_{t-1}}{dU}$$

Moving from derivatives to partial derivatives

RNN gradients

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Oh_t + b_o$$

$$L_t = -\log p(x_t | x_{<t}) = -\log \text{softmax}(o_t)_{x_t}$$

$$L = \sum_{t=1}^m L_t$$

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \frac{dh_t}{dU}$$

$$\begin{aligned} \frac{dh_t}{dU} &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \boxed{\frac{dh_{t-1}}{dU}} = \\ &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \boxed{\left(\frac{\partial h_{t-1}}{\partial U} + \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{dh_{t-2}}{dU} \right)} \end{aligned}$$

RNN gradients

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Oh_t + b_o$$

$$L_t = -\log p(x_t | x_{<t}) = -\log \text{softmax}(o_t)_{x_t}$$

$$L = \sum_{t=1}^m L_t$$

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \frac{dh_t}{dU}$$

$$\begin{aligned} \frac{dh_t}{dU} &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{dh_{t-1}}{dU} = \\ &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \left(\frac{\partial h_{t-1}}{\partial U} + \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{dh_{t-2}}{dU} \right) = \\ &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{dh_{t-2}}{dU} \end{aligned}$$

RNN gradients

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Oh_t + b_o$$

$$L_t = -\log p(x_t | x_{<t}) = -\log \text{softmax}(o_t)_{x_t}$$

$$L = \sum_{t=1}^m L_t$$

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \frac{dh_t}{dU}$$

$$\begin{aligned} \frac{dh_t}{dU} &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{dh_{t-1}}{dU} = \\ &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \left(\frac{\partial h_{t-1}}{\partial U} + \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{dh_{t-2}}{dU} \right) = \\ &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{dh_{t-2}}{dU} = \\ &= \sum_{k=1}^t \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial U} \end{aligned}$$

RNN gradients

$$h_t = \sigma(Wx_t + Uh_{t-1} + b_h)$$

$$o_t = Oh_t + b_o$$

$$L_t = -\log p(x_t | x_{<t}) = -\log \text{softmax}(o_t)_{x_t}$$

$$L = \sum_{t=1}^m L_t$$

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \frac{dh_t}{dU}$$

$$\begin{aligned} \frac{dh_t}{dU} &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{dh_{t-1}}{dU} = \\ &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \left(\frac{\partial h_{t-1}}{\partial U} + \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{dh_{t-2}}{dU} \right) = \\ &= \frac{\partial h_t}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial U} + \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{dh_{t-2}}{dU} = \\ &= \sum_{k=1}^t \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial U} \end{aligned}$$

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \left[\sum_{k=1}^t \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial U} \right]$$

Gradient explosion

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \left[\sum_{k=1}^t \underbrace{\left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right)}_{\substack{\text{Chain of derivatives} \\ \text{multiplication}}} \frac{\partial h_k}{\partial U} \right]$$

$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| > 1$$

- Gradient $\frac{dL}{dU}$ explodes
- Model diverges, NaNs in weights

Gradient explosion

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \left[\sum_{k=1}^t \underbrace{\left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right)}_{\substack{\text{Chain of derivatives} \\ \text{multiplication}}} \frac{\partial h_k}{\partial U} \right]$$

$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| > 1$$

- Gradient $\frac{dL}{dU}$ explodes
- Model diverges, NaNs in weights

Solutions:

- Regularization
- Reducing learning rate
- Gradient clipping

Gradient explosion

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \left[\sum_{k=1}^t \underbrace{\left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right)}_{\text{Chain of derivatives multiplication}} \frac{\partial h_k}{\partial U} \right]$$

$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| > 1$$

- Gradient $\frac{dL}{dU}$ explodes
- Model diverges, NaNs in weights

Solutions:

- Regularization
- Reducing learning rate
- Gradient clipping

$$1. g \leftarrow \min \left(1, \frac{\text{max norm}}{\|g\|} \right) \cdot g$$

Correct way

$$2. g \leftarrow \text{clip}(g, -C, C)$$

← Lazy way (changes gradient direction)

Gradient vanishing

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \left[\sum_{k=1}^t \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial U} \right]$$

$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| < 1$$

- Gradient vanishes
- The model stops training
- The model does not capture distant dependencies!

Gradient vanishing

$$\frac{dL}{dU} = \sum_{t=1}^m \frac{dL_t}{do_t} \frac{do_t}{dh_t} \left[\sum_{k=1}^t \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial U} \right]$$

$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| < 1$$

- Gradient vanishes
- The model stops training
- The model does not capture distant dependencies!

Gradient decay is a common problem with RNNs.

It cannot be fixed with tricks.

Why gradient vanishes?

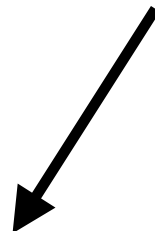
$$h_j = \sigma(\underbrace{Wx_j + Uh_{j-1} + b_h}_{z_j})$$

$$\frac{\partial h_j}{\partial h_{j-1}} = \frac{\partial \sigma(z_j)}{\partial h_{j-1}} = \frac{\partial \sigma(z_j)}{\partial z_j} \frac{\partial z_j}{\partial h_{j-1}}$$

Why gradient vanishes?

$$h_j = \sigma(\underbrace{Wx_j + Uh_{j-1} + b_h}_{z_j})$$

Element-wise multiplication

$$\frac{\partial h_j}{\partial h_{j-1}} = \frac{\partial \sigma(z_j)}{\partial h_{j-1}} = \frac{\partial \sigma(z_j)}{\partial z_j} \frac{\partial z_j}{\partial h_{j-1}} = \left(\sigma(z_j) \odot (1 - \sigma(z_j)) \right) U$$


Why gradient vanishes?

$$h_j = \sigma(\underbrace{Wx_j + Uh_{j-1} + b_h}_{z_j})$$

Element-wise multiplication

$$\frac{\partial h_j}{\partial h_{j-1}} = \frac{\partial \sigma(z_j)}{\partial h_{j-1}} = \frac{\partial \sigma(z_j)}{\partial z_j} \frac{\partial z_j}{\partial h_{j-1}} = \left(\sigma(z_j) \odot (1 - \sigma(z_j)) \right) U$$

Let's look at the spectral norm

$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| \leq \underbrace{\| \sigma(z_j) \odot (1 - \sigma(z_j)) \|}_{< 1} \cdot \|U\|$$

т. к. $\sigma(z) \in [0,1]$

Why gradient vanishes?

$$h_j = \sigma(\underbrace{Wx_j + Uh_{j-1} + b_h}_{z_j})$$

Element-wise multiplication

$$\frac{\partial h_j}{\partial h_{j-1}} = \frac{\partial \sigma(z_j)}{\partial h_{j-1}} = \frac{\partial \sigma(z_j)}{\partial z_j} \frac{\partial z_j}{\partial h_{j-1}} = \left(\sigma(z_j) \odot (1 - \sigma(z_j)) \right) U$$

Let's look at the spectral norm

$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| \leq \underbrace{\| \sigma(z_j) \odot (1 - \sigma(z_j)) \|}_{< 1} \cdot \|U\|$$

т. к. $\sigma(z) \in [0, 1]$

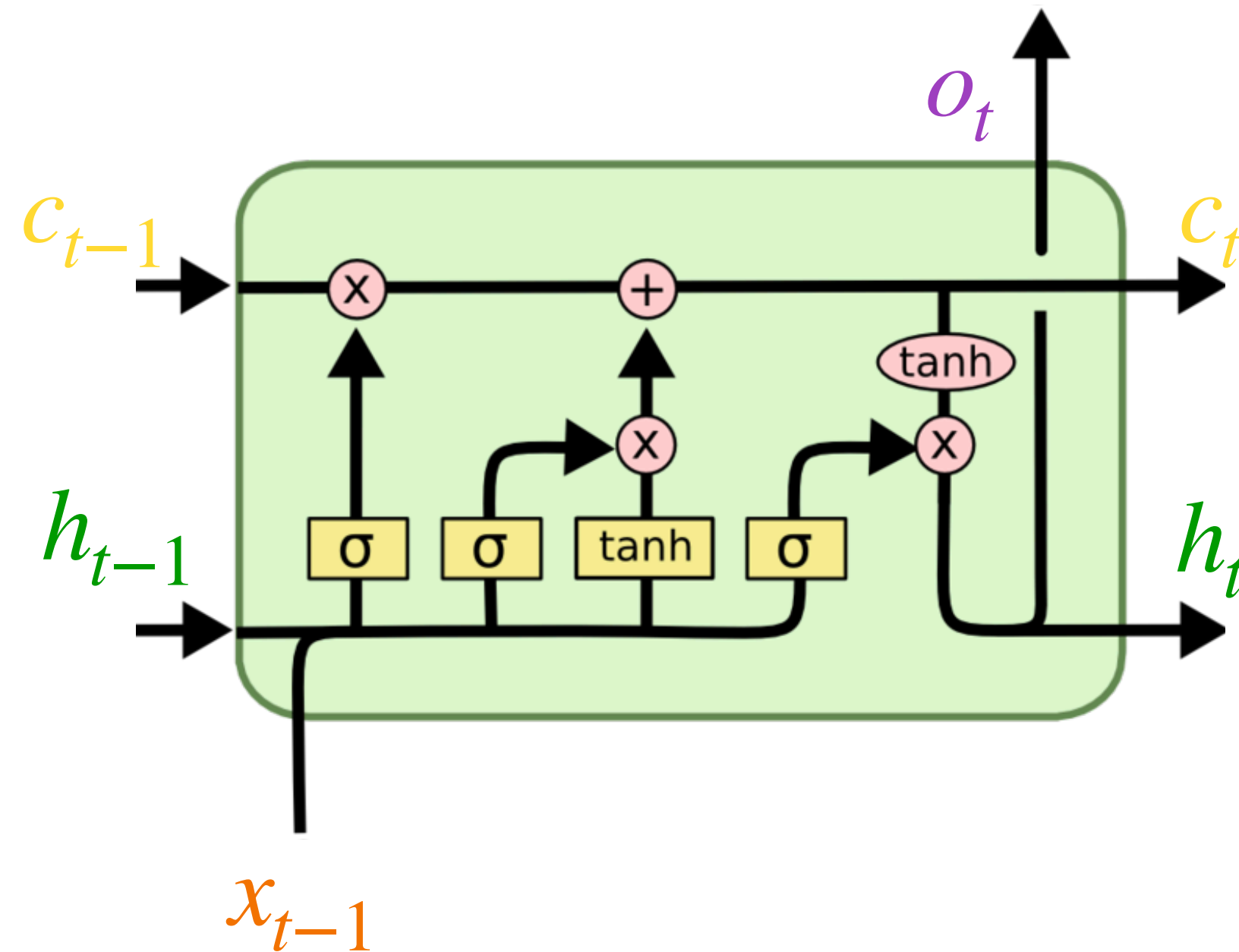
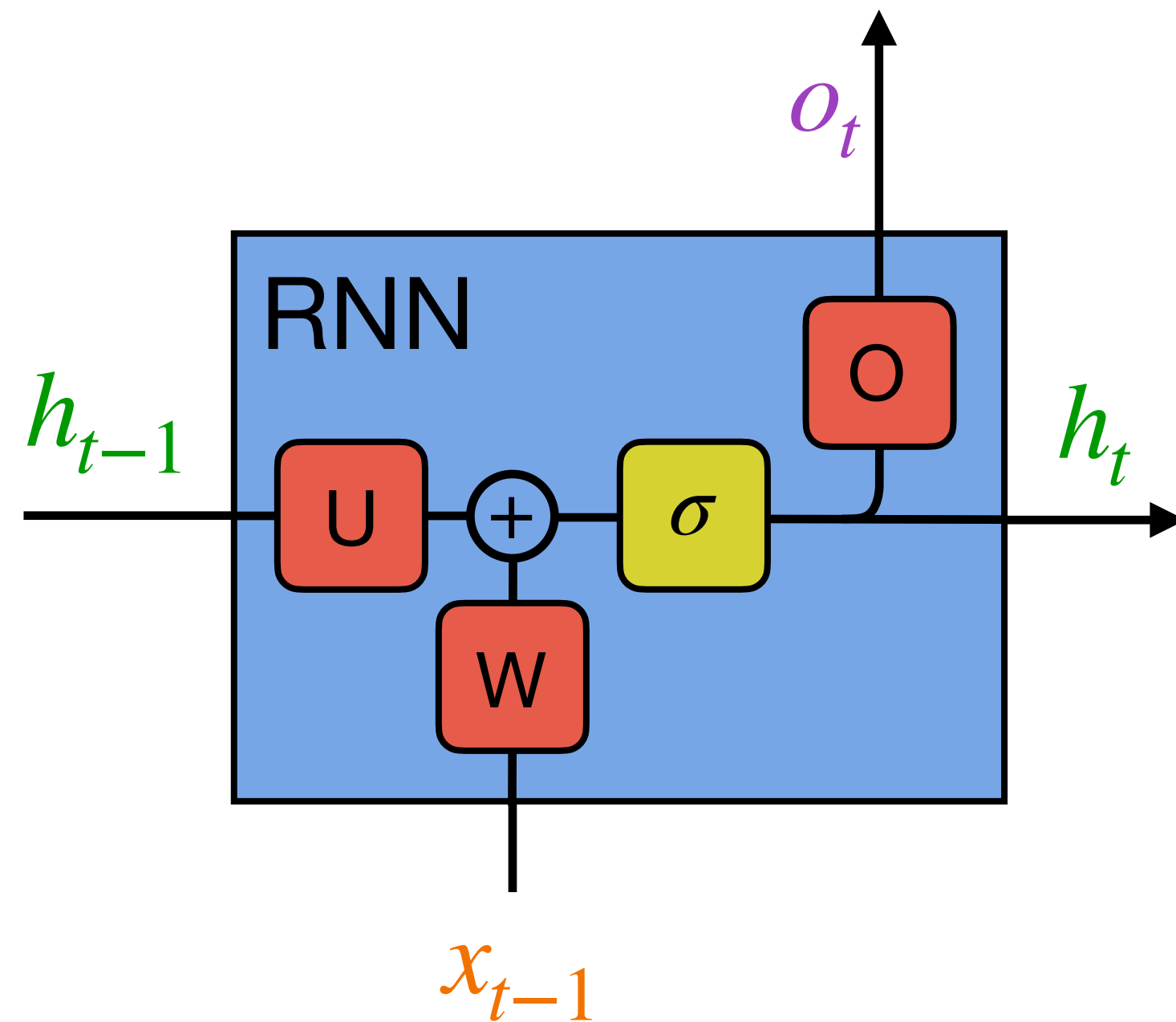
If U is orthogonal, then

$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| \leq \| \sigma(z_j) \odot (1 - \sigma(z_j)) \| < 1$$

Long shot-term memory (LSTM)

A memory cell c_t is added to LSTM

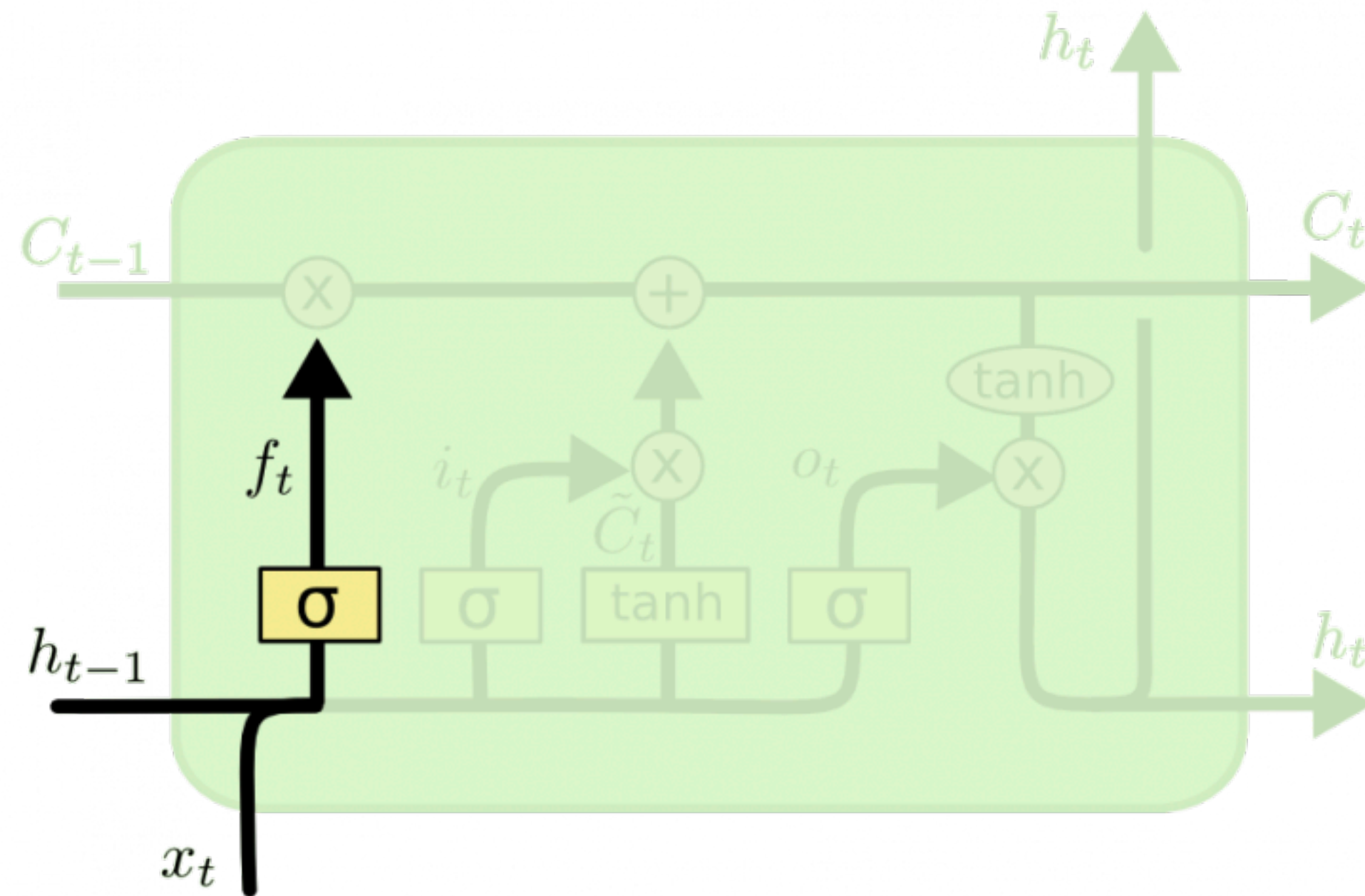
Thanks to it, the model does not forget old information.



No activations are applied to c_t . Gradients don't vanish

LSTM: forget gate

Controls what information should be forgotten and what should be left.

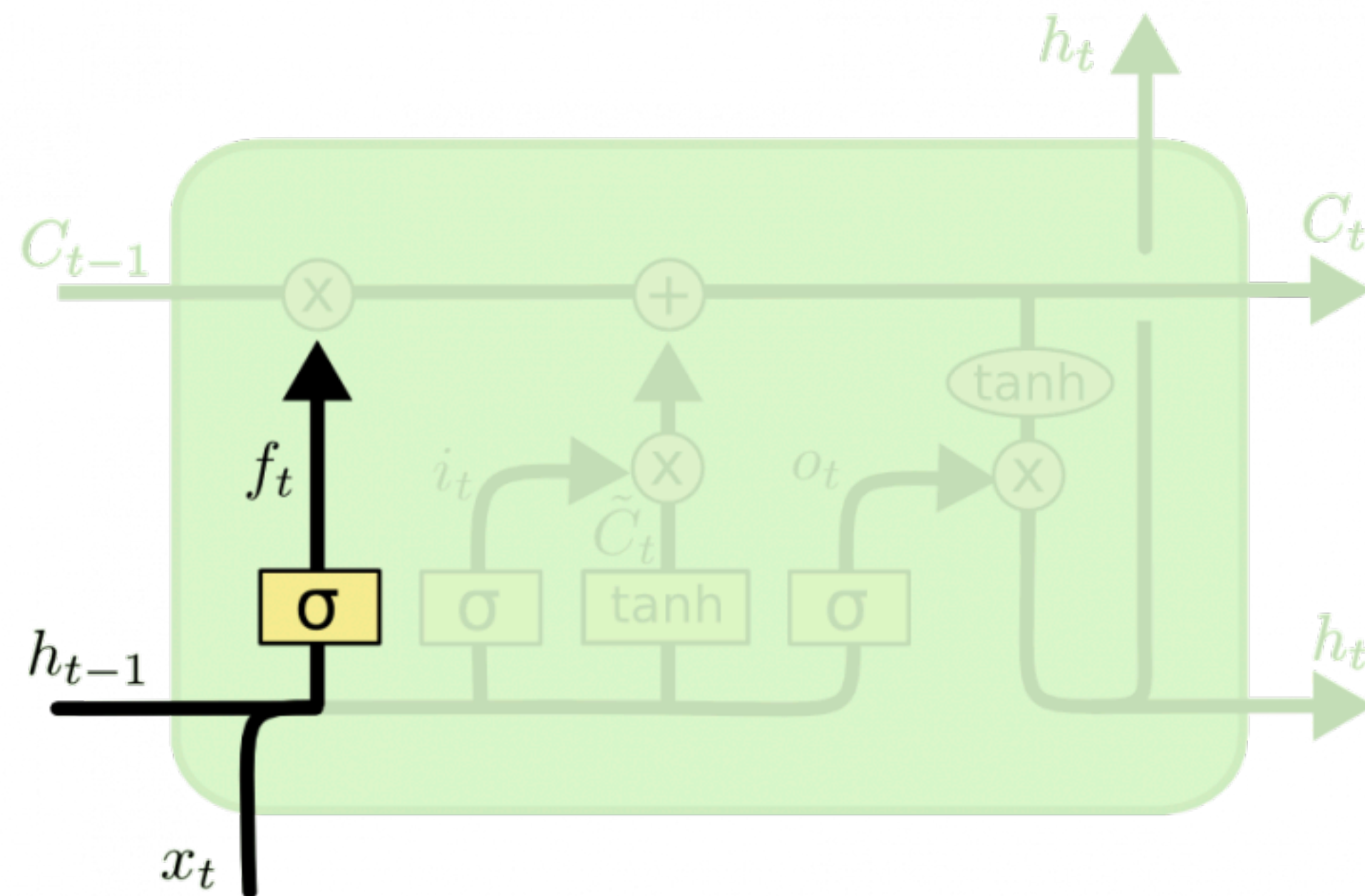


$$f_t = \sigma(W_f x_{t-1} + U_f h_{t-1} + b_f)$$

$$f_t \in [0,1]$$

LSTM: forget gate

Controls what information should be forgotten and what should be left.



$$f_t = \sigma(W_f x_{t-1} + U_f h_{t-1} + b_f)$$

$$f_t \in [0, 1]$$

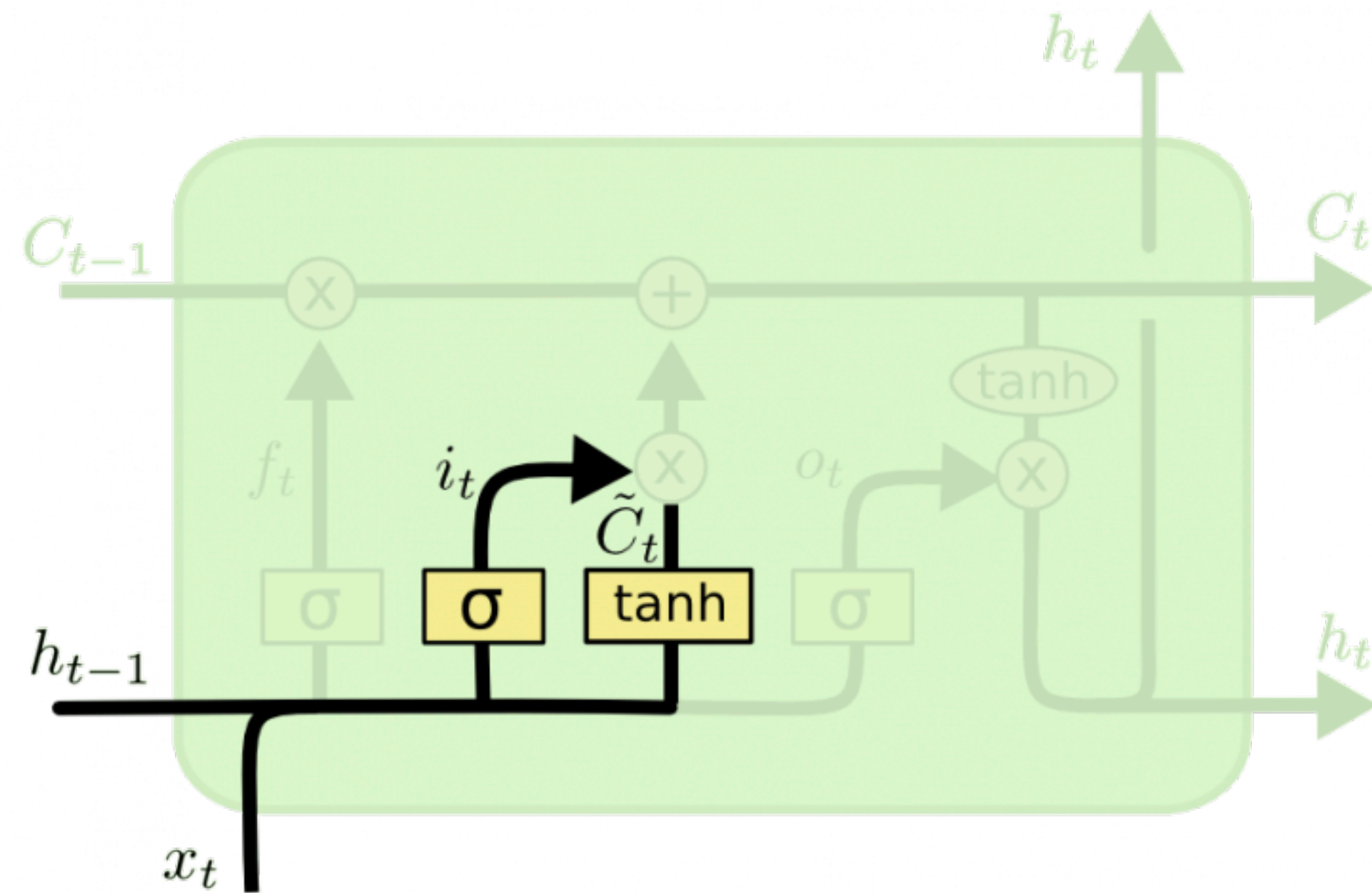
The food was **tasty** and we were not disappointed.

x_3 – marker word, we can forget everything before it

x_7 – негативный маркер, но до него идет "не".
Знаем об этом из h_7 .

LSTM: input gate

Controls what information should be added to the memory cell c_t



Because of i_t we can avoid adding

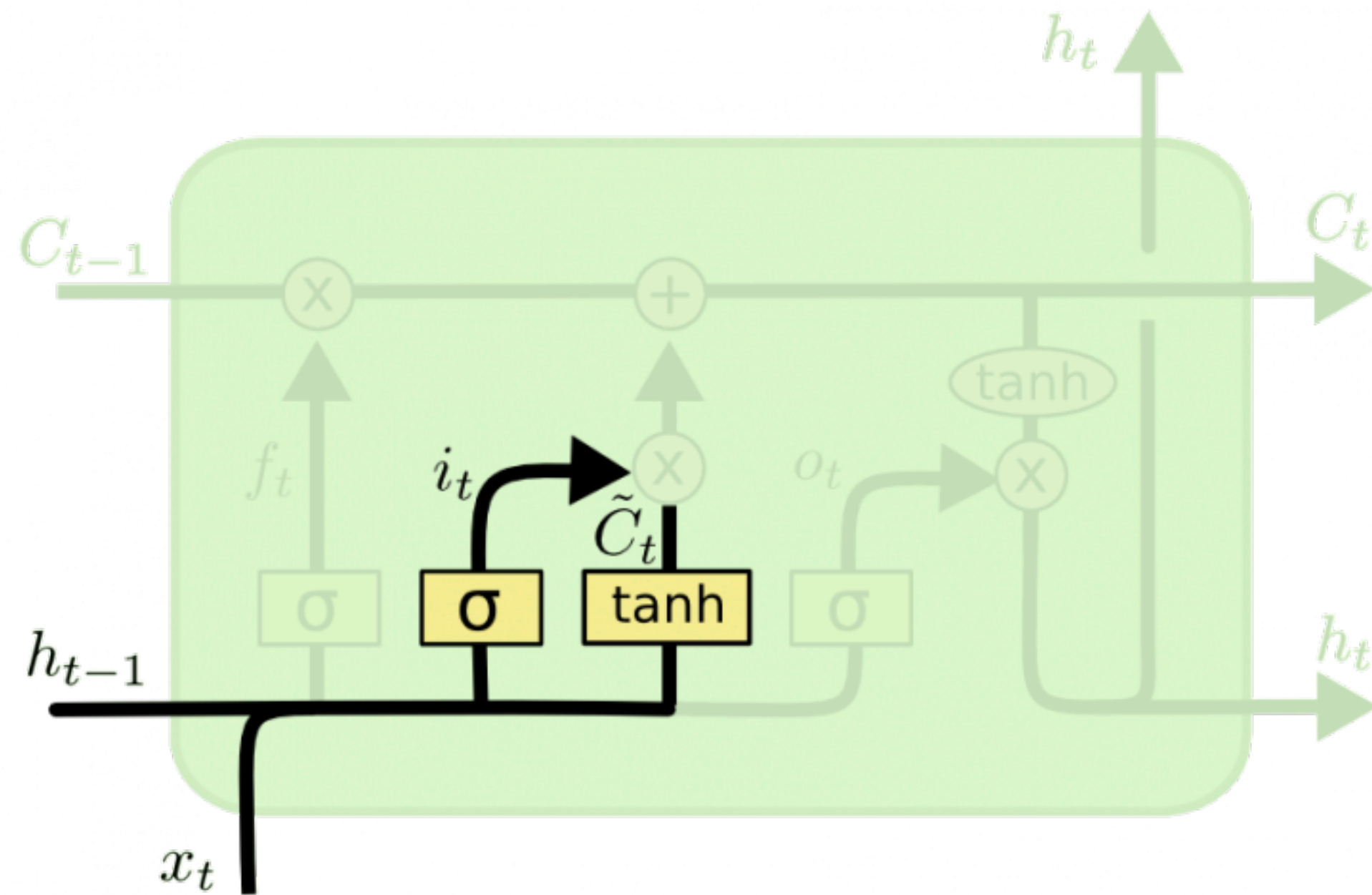
$$i_t = \sigma(W_i x_{t-1} + U_i h_{t-1} + b_i)$$

$$i_t \in [0, 1]$$

$$\tilde{c}_t = \tanh(W_i x_{t-1} + U_i h_{t-1} + b_i)$$

LSTM: input gate

Controls what information should be added to the memory cell c_t



Because of i_t we can avoid adding

$$i_t = \sigma(W_i x_{t-1} + U_i h_{t-1} + b_i)$$

$$i_t \in [0, 1]$$

$$\tilde{c}_t = \tanh(W_i x_{t-1} + U_i h_{t-1} + b_i)$$

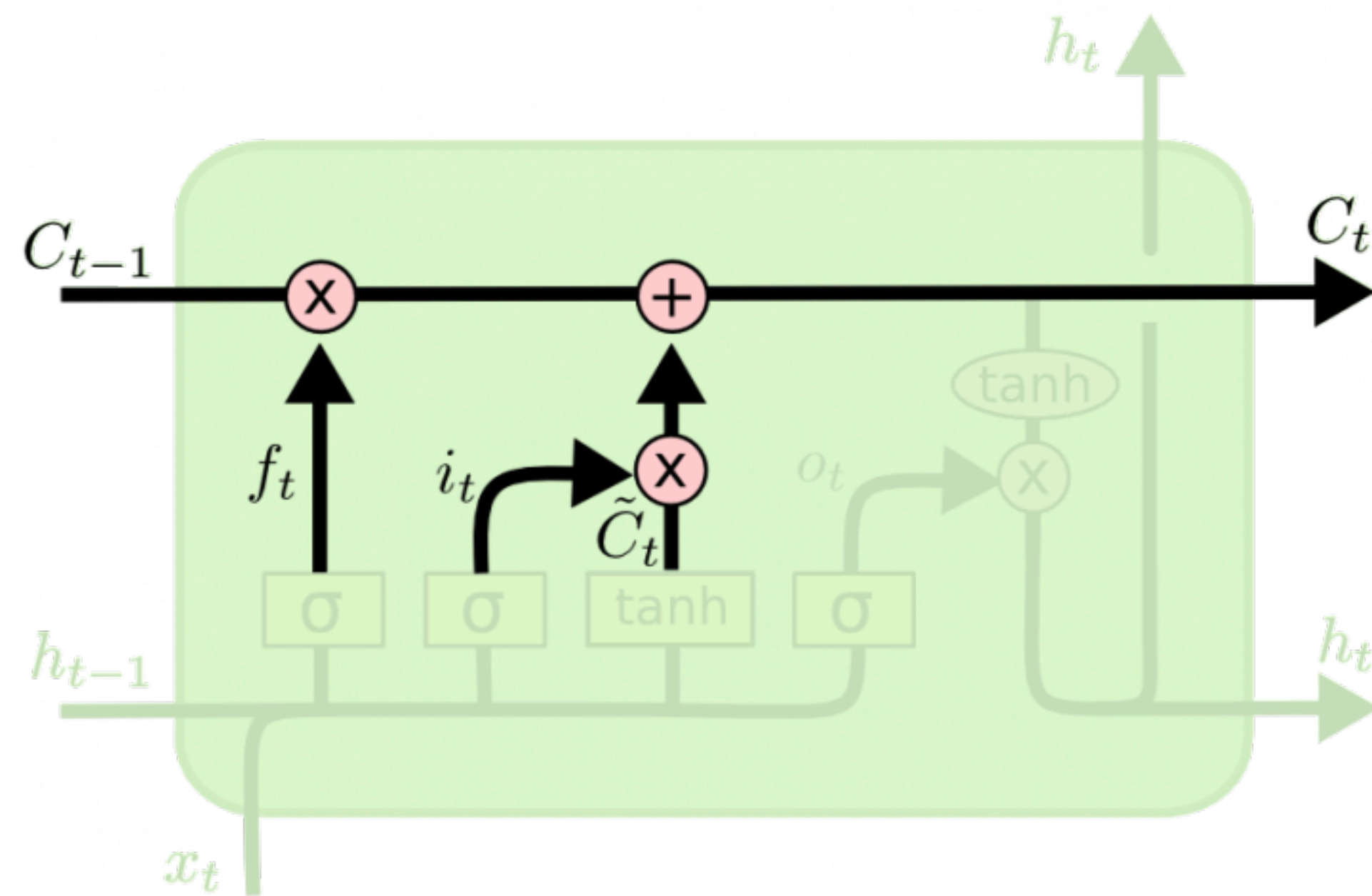
The food was **tasty** and we were **not disappointed**.

x_3 – marker word.
Remember that the
class is positive.

x_5 – negative marker, but
before it comes "**not**". We
know about this from h_t

LSTM: memory update

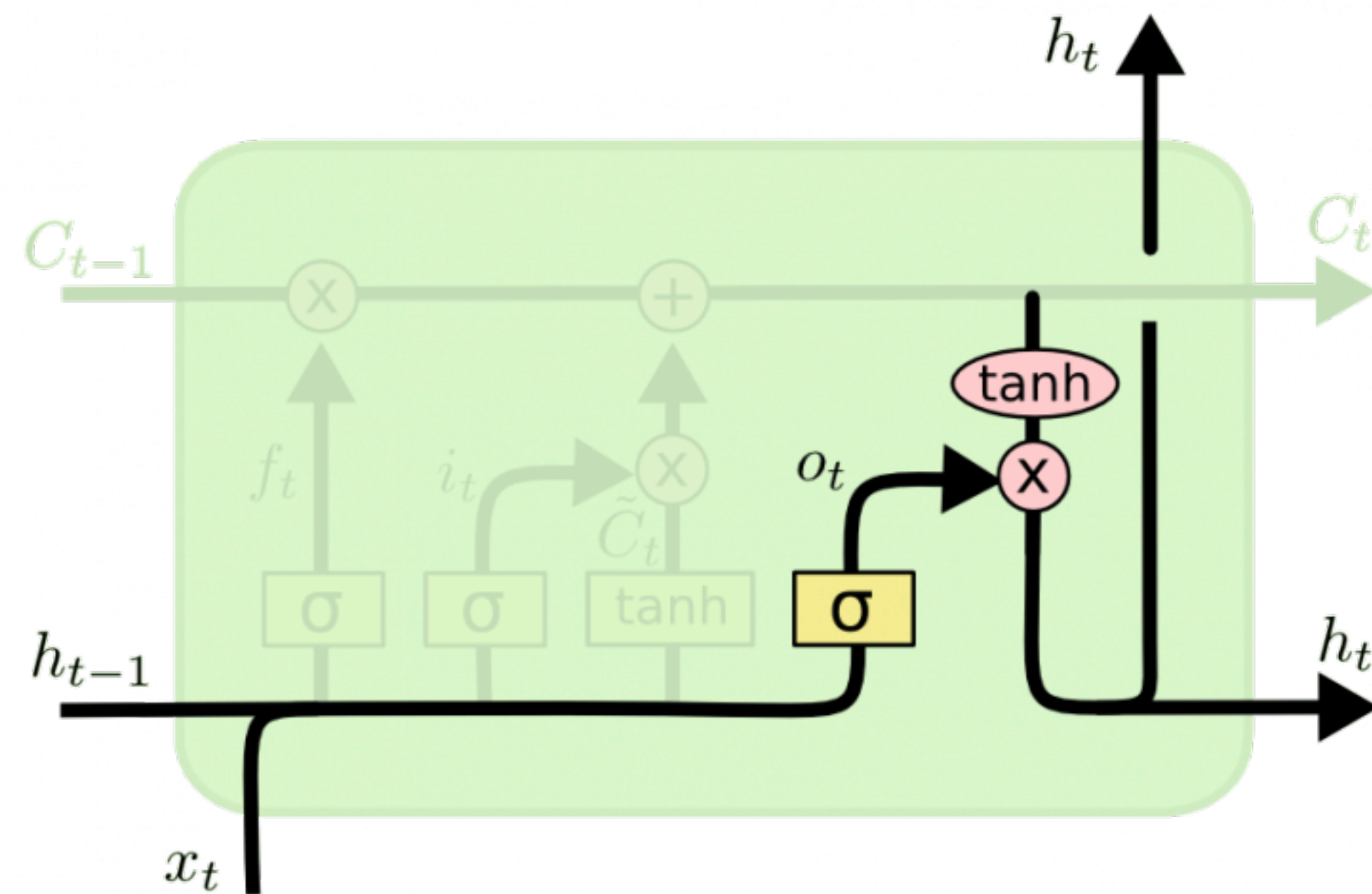
Remove unnecessary information and add new information.



$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

LSTM: output gate

Controls the output of the current step.

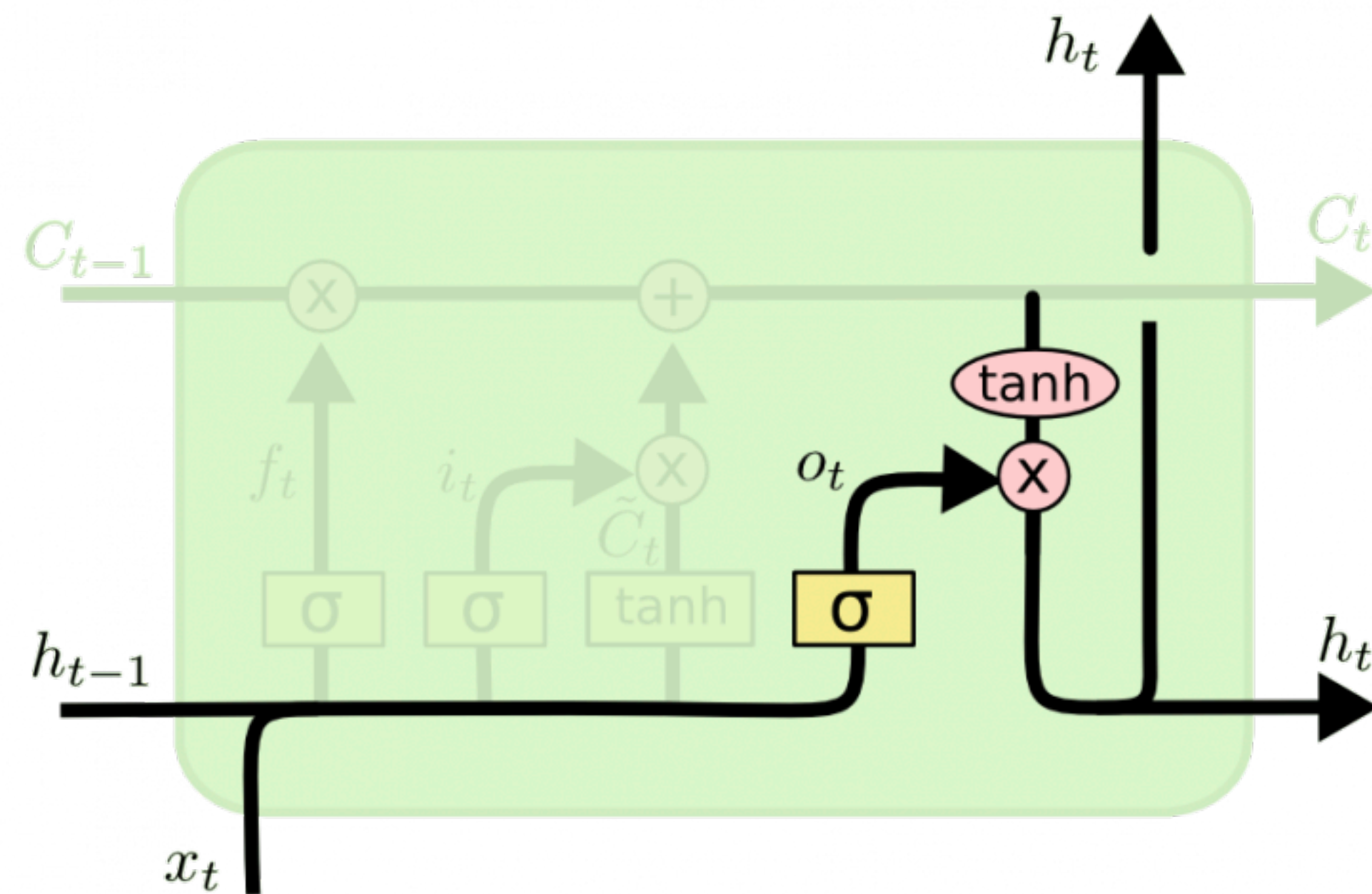


$$o_t = \sigma(W_t x_{t-1} + U_t h_{t-1} + b_t)$$

$$h_t = o_t \odot \tanh(c_t)$$

LSTM: output gate

Controls the output of the current step.



$$o_t = \sigma(W_t x_{t-1} + U_t h_{t-1} + b_t)$$

$$h_t = o_t \odot \tanh(c_t)$$

The **teacher** is teaching a **lesson** on recurrent models. _

Beginning of a new sentence. We must remember that we are talking about a **teacher** and a **lesson**.

The most important

Classification

- Text/word classification is the most popular task in NLP.
- Simple methods often perform well.
- Quality directly depends on how well the features are extracted.

Text generation

- Classification with a huge number of classes
- This is significantly more difficult, as it requires a more precise understanding of the text.
- Simple models never cope.